Yes. We can change the backend to use **FAISS only** and **Gemini 2.5 Flash only**.

So the final stack will be:

```
Python
Flask
PyPDF
LangChain
FAISS               ← Vector Database
SentenceTransformers ← Embeddings
Gemini 2.5 Flash    ← LLM
```

We will **remove ChromaDB completely**.

One important distinction:

```
FAISS ≠ Embedding Model ≠ LLM
```

They have different jobs:

```
PDF
 ↓
Text Extraction
 ↓
Chunking
 ↓
Embedding Model
 ↓
FAISS
 ↓
Retriever
 ↓
Gemini 2.5 Flash
 ↓
Answer
```

For Gemini, use your API key in `.env` rather than hard-coding it.

1. Updated project structure

Use this:

```
real_estate_rag/
│
├── app.py
├── requirements.txt
├── .env
├── .gitignore
│
├── documents/
│   │
│   ├── property_A/
│   │   ├── property_details.pdf
│   │   ├── rental_agreement.pdf
│   │   └── inspection_report.pdf
│   │
│   └── property_B/
│       ├── property_details.pdf
│       ├── rental_agreement.pdf
│       └── inspection_report.pdf
│
├── vector_db/
│   └── faiss_index/
│
├── rag/
│   ├── document_loader.py
│   ├── text_splitter.py
│   ├── embeddings.py
│   ├── vector_store.py
│   ├── retriever.py
│   └── prompts.py
│
├── features/
│   ├── qa.py
│   ├── summarization.py
│   └── comparison.py
```

```
│
└── utils/
    └── preprocessing.py
```

The important change is:

```
vector_db/
└── faiss_index/
```

instead of:

```
vector_db/
└── chroma_db/
```

2. Install the correct packages

Remove the Chroma-related packages from your project.

Install:

```
pip install flask
pip install flask-cors
pip install python-dotenv
pip install pypdf
pip install langchain
pip install langchain-community
pip install langchain-text-splitters
pip install faiss-cpu
pip install sentence-transformers
pip install google-generativeai
```

Then:

```
pip freeze > requirements.txt
```

You **do not need**:

```
chromadb
langchain-chroma
```

for this project.

3. .env

Create:

`.env`

Use:

```
GEMINI_API_KEY=your_gemini_api_key_here
```

Do not write:

```
GEMINI_API_KEY = "actual-key"
```

inside your Python files.

4. .gitignore

Create:

`.gitignore`

Put:

```
venv/
.env
__pycache__/
vector_db/
*.pyc
```

This prevents your Gemini API key and FAISS files from accidentally going to GitHub.

5. Step 1 — PDF loader (extract the txt)

Keep:

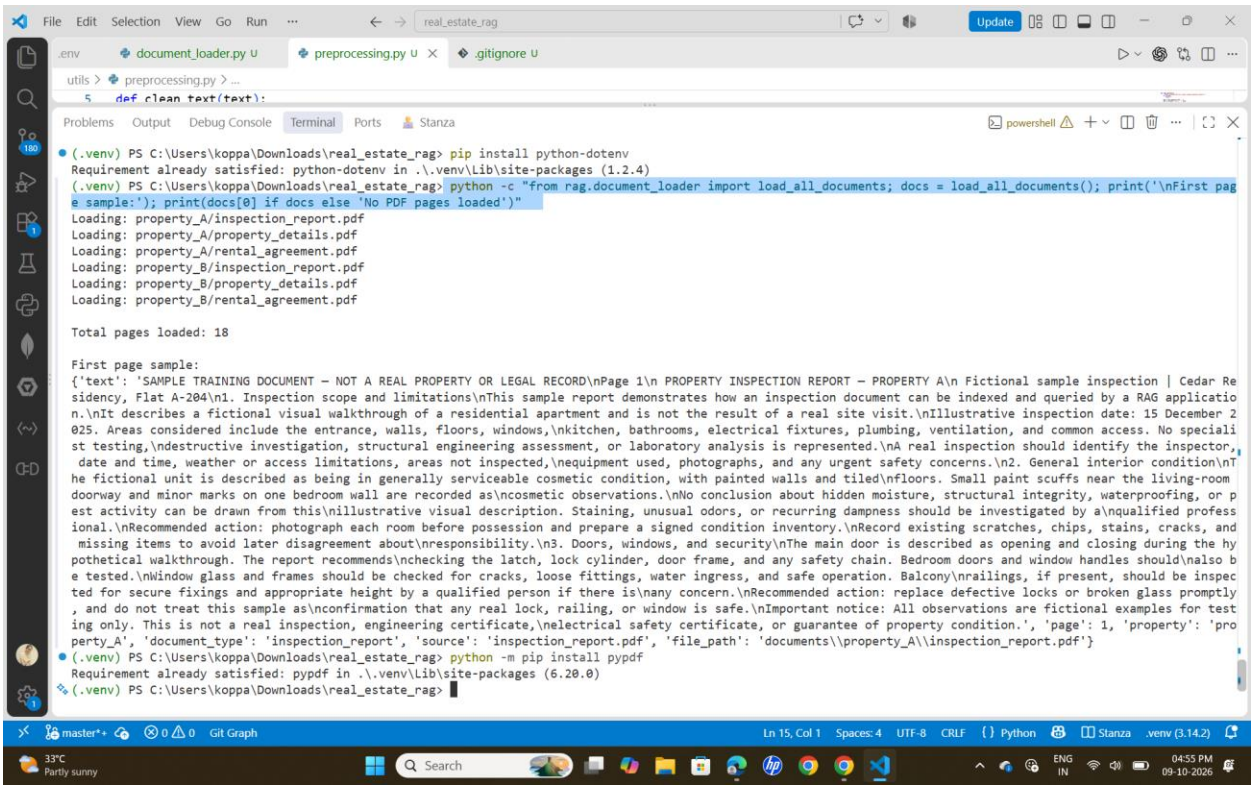
rag/document_loader.py

Use:

```
import osfrom pypdf import PdfReaderfrom utils.preprocessing import
clean_textdef load_pdf(file_path):    reader = PdfReader(file_path)
pages = []    for page_number, page in enumerate(        reader.pages,
start=1    ):        text = page.extract_text()        if text:
text = clean_text(text)
pages.append({                "text": text,                "page":
page_number            })    return pagesdef
load_all_documents(documents_folder="documents"):    all_documents =
[]    if not os.path.exists(documents_folder):
```

Output:

```
python -c "from rag.document_loader import load_all_documents; docs =
load_all_documents(); print('\nFirst page sample:'); print(docs[0] if
docs else 'No PDF pages loaded')"
```



What you have completed

PDF loading successful:

All 6 PDF files were found and processed.

18 pages extracted:

The loader extracted text from all 18 pages.

Preprocessing working:

The extracted text is passed through `clean_text()`

6. Step 2 — Text preprocessing

`utils/preprocessing.py`

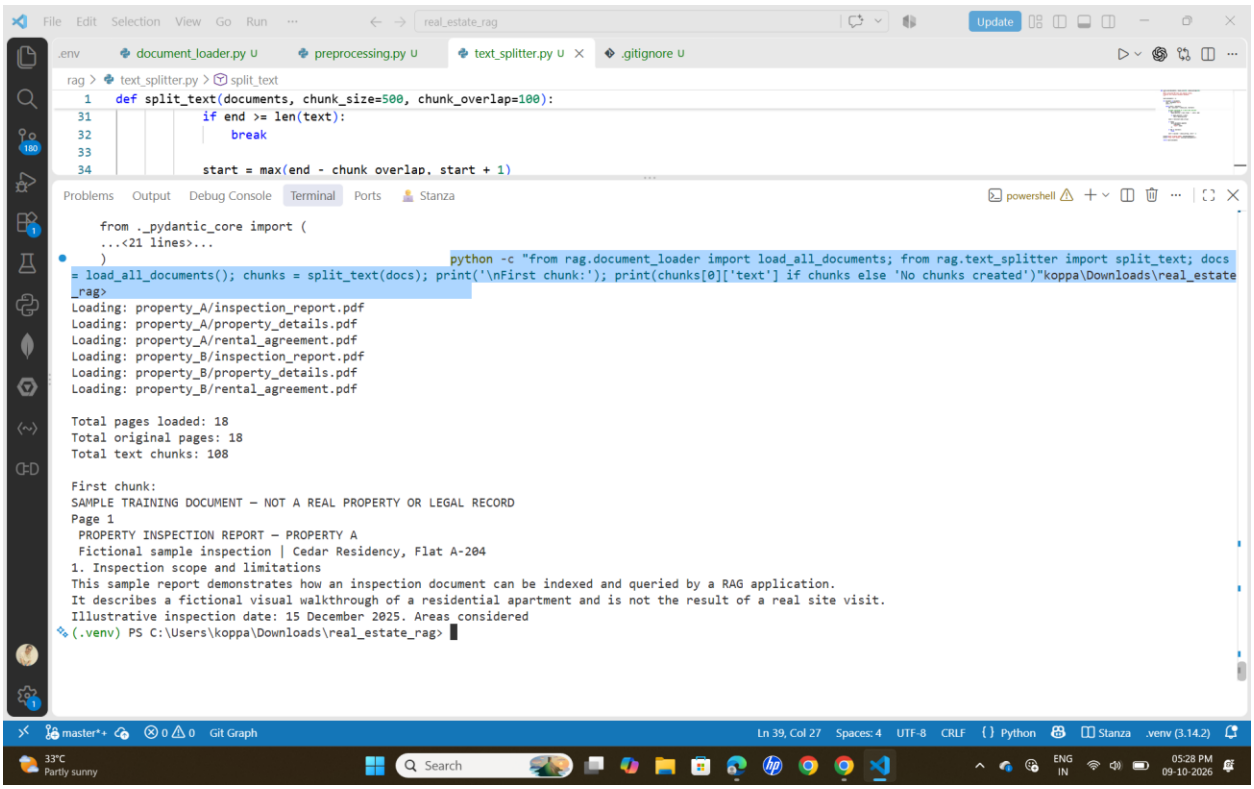
```
import redef clean_text(text):    text = re.sub(        r"\s+",
" ",        text    )    return text.strip()
```

7. Step 3 — Chunking

rag/text_splitter.py

```
from langchain_text_splitters import
(    RecursiveCharacterTextSplitter)def split_documents(documents):
splitter = RecursiveCharacterTextSplitter(        chunk_size=800,
chunk_overlap=100    )    chunks = []    for document in documents:
split_texts =
splitter.split_text(            document["text"]        )        for
chunk in split_texts:
chunks.append({                "text": chunk,
"metadata": {                  "property": document["property"],
"source": document["source"],                    "page":
document["page"]                }            })    return chunks
```

OUTPUT:



Your Step 2: text_splitter.py is working correctly! ✅ Your screenshot shows that all 18 pages were loaded and split into 108 text chunks.

Why did we create text_splitter.py?

Your Real Estate Document Intelligence project needs to answer questions from property PDFs, such as:

- What is the monthly rent for Property A?
- What is the security deposit?
- What are the tenant's maintenance responsibilities?
- Compare Property A and Property B.

If we give all the PDF text to the AI for every question, it can be inefficient and difficult to find the exact information needed. So, we split the documents into smaller pieces called chunks.

How your project works

Step 1: PDF Loader

Extract text from 18 PDF pages

Step 2: Text Splitter

18 pages → 108 chunks

Each chunk is approximately 500 characters or fewer

Step 3: Embeddings

Convert text chunks into numerical vectors

Step 4: FAISS

Store vectors for similarity search

What do chunk_size and chunk_overlap mean?

Your code uses:

chunk_size = 500chunk_overlap = 100

Imagine your PDF contains 1,000 characters.

- Chunk 1: characters 1–500
- Chunk 2: starts near character 401 and continues for up to 500 characters.
- Overlap: approximately 100 characters are shared between neighboring chunks, helping preserve context.

Your current implementation prefers splitting at spaces, so the actual chunk sizes and overlaps can vary slightly.

What does 108 chunks mean?

It means your program created 108 smaller text sections from the 18 extracted PDF pages. Each chunk also retains its property name, page number, and source PDF, which will help you identify where a retrieved answer came from.

Next step — Step 3: Embeddings. We will convert these 108 text chunks into numerical vectors, then use FAISS only as your vector database. We will not use ChromaDB.

8. Step 4 — Embeddings

Here we're using a local embedding model.

rag/embeddings.py

```
from langchain_community.embeddings import
(    HuggingFaceEmbeddings)def get_embedding_model():    return
HuggingFaceEmbeddings(        model_name="sentence-transformers/all-
MiniLM-L6-v2"    )
```

The process is:

```
Text
 ↓
all-MiniLM-L6-v2
 ↓
Vector
```

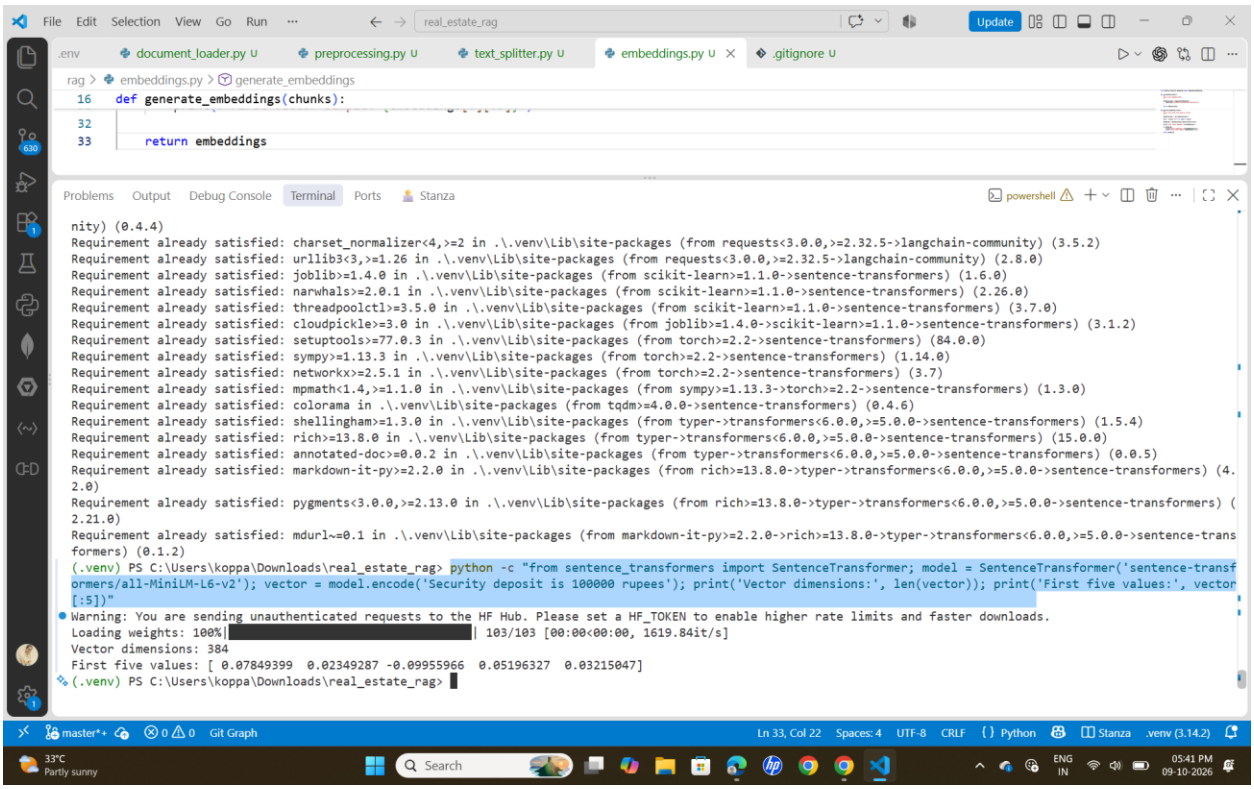
Example:

```
"Security deposit is ₹1,00,000"

        ↓

[0.124, -0.283, 0.512, ...]
```

Output:



What your output means:

Model loaded successfully

`sentence-transformers/all-MiniLM-L6-v2`

Vector dimensions: 384

Each text is represented by 384 numerical values.

Local embedding model is running

The model downloaded its weights and generated the embedding on your machine.

9. Step 5 — FAISS vector database

This is the major change from the previous version.

Create:

rag/vector_store.py

Use:

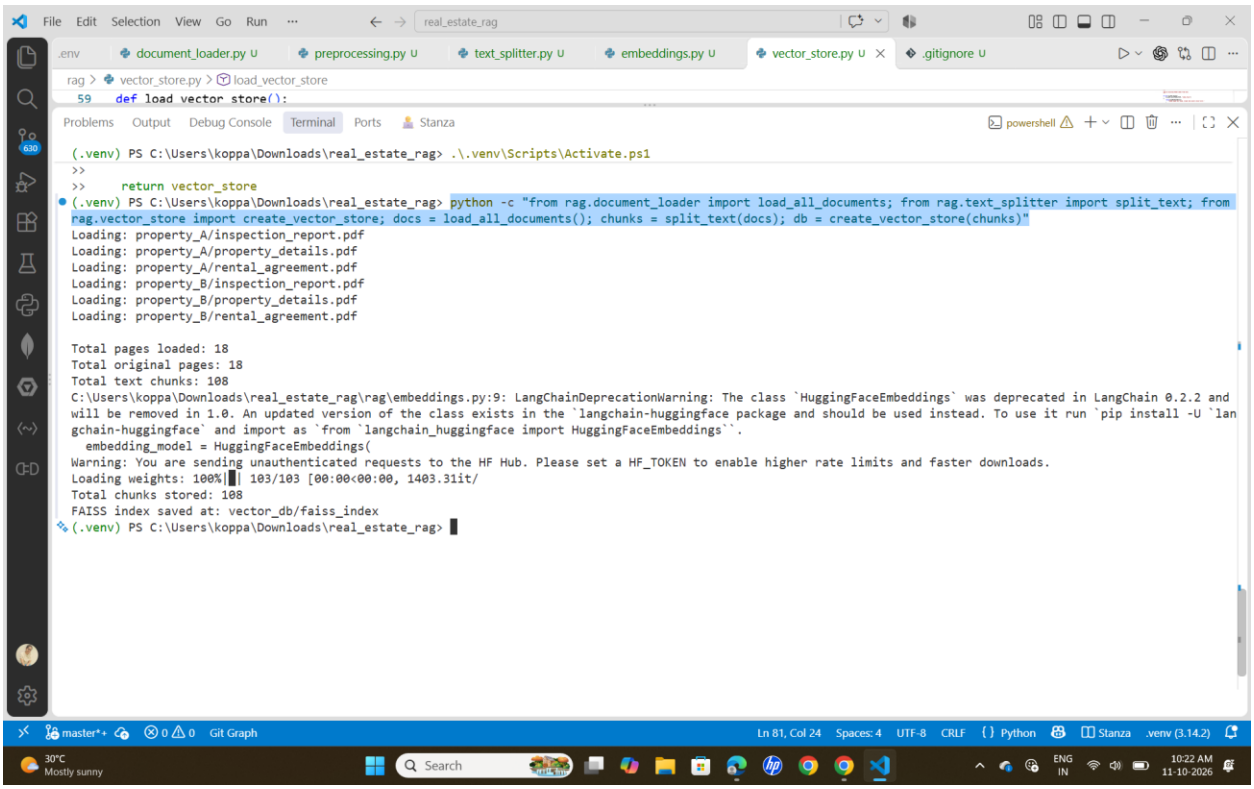
```
import osfrom langchain_community.vectorstores import FAISSfrom
langchain_core.documents import Documentfrom rag.embeddings import
get_embedding_modelFAISS_PATH = "vector_db/faiss_index"def
create_vector_store(chunks):    embedding_model =
get_embedding_model()    documents = []    for chunk in chunks:
document = Document(            page_content=chunk["text"],
metadata=chunk["metadata"]        )        documents.append(document)
vector_store = FAISS.from_documents(        documents,
embedding_model    )    os.makedirs(        "vector_db",
exist_ok=True    )
```

Now the architecture is:

```
Chunks
   ↓
Embeddings
   ↓
FAISS
   ↓
vector_db/faiss_index/
```

Output:

```
python -c "from rag.document_loader import load_all_documents; from
rag.text_splitter import split_text; from rag.vector_store import
create_vector_store; docs = load_all_documents(); chunks =
split_text(docs); db = create_vector_store(chunks)"
```



PDF loading completed:

18 pages extracted from 6 PDF files.

Text splitting completed:

108 text chunks created.

FAISS index created:

All 108 chunks stored in the vector index.

Index saved locally:

`vector_db/faiss_index/`

10. What gets stored in FAISS?

Conceptually:

```
FAISS
│
├── Vector 1
│    └── Property A / rental agreement / page 1
│
├── Vector 2
│    └── Property A / rental agreement / page 2
│
├── Vector 3
│    └── Property B / rental agreement / page 1
│
└── Vector 4
     └── Property B / inspection report / page 2
```

The metadata is associated with each document.

This is what will allow us to eventually distinguish:

`Property A`

from:

`Property B`

11. Step 6 — Retriever

`rag/retriever.py`

```
from rag.vector_store import load_vector_storedef
retrieve_documents(    question,    k=4):    vector_store =
load_vector_store()    documents =
vector_store.similarity_search(        question,        k=k    )
return documents
```

User:

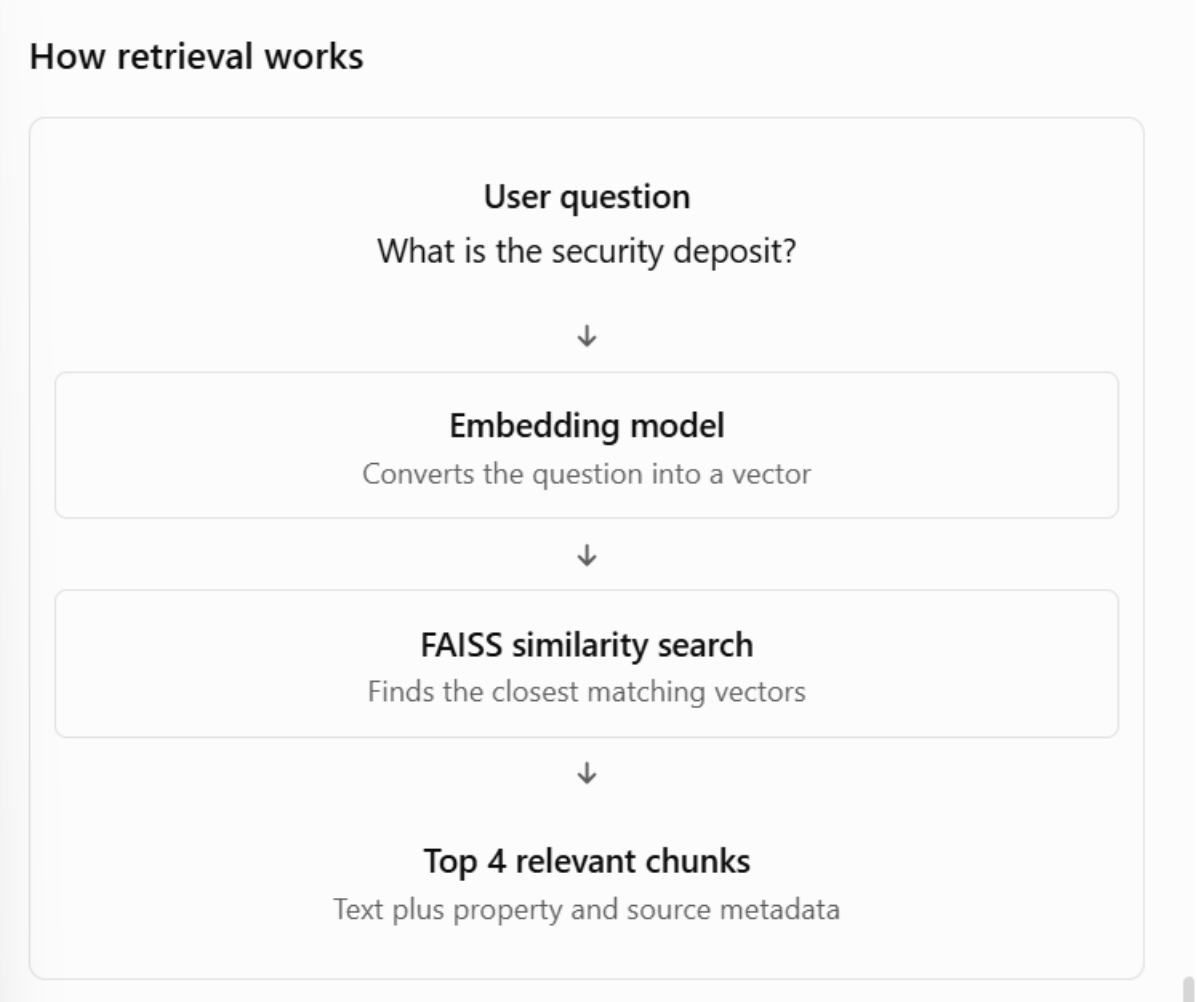
`What is the security deposit?`

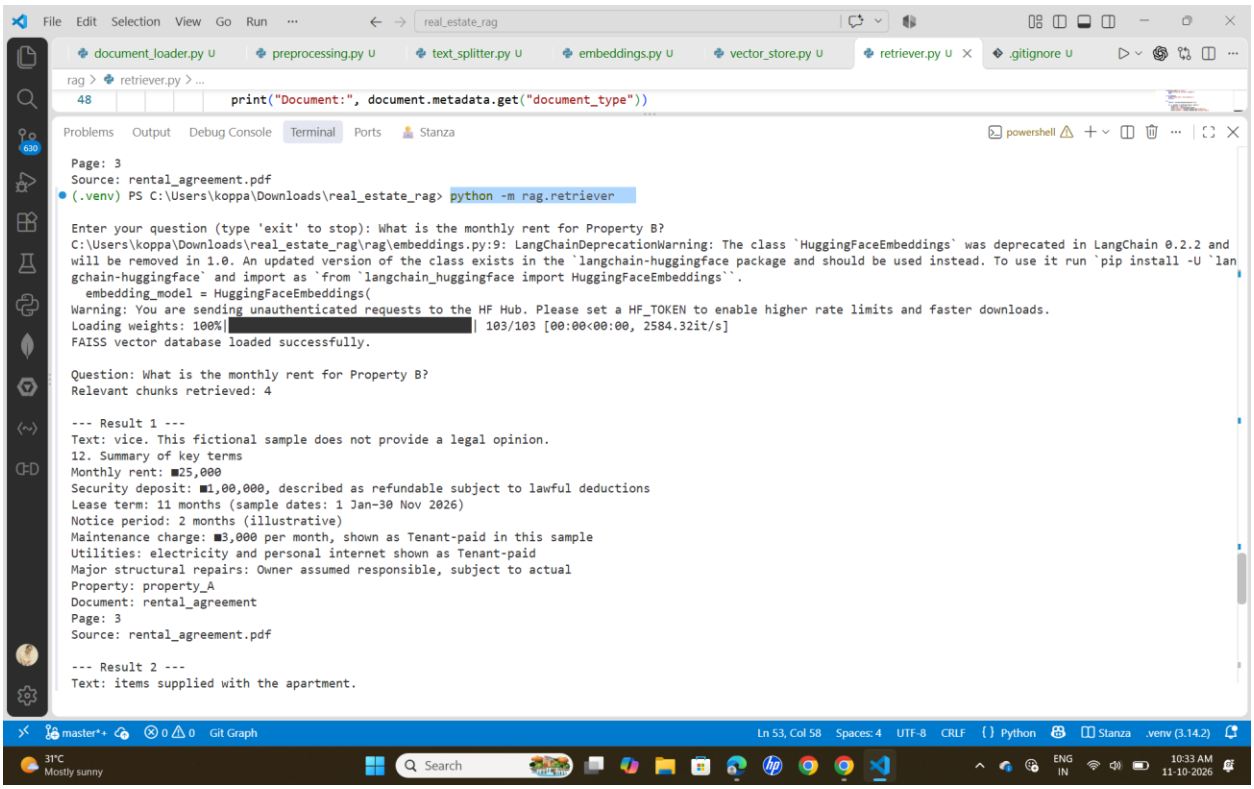
Retriever:

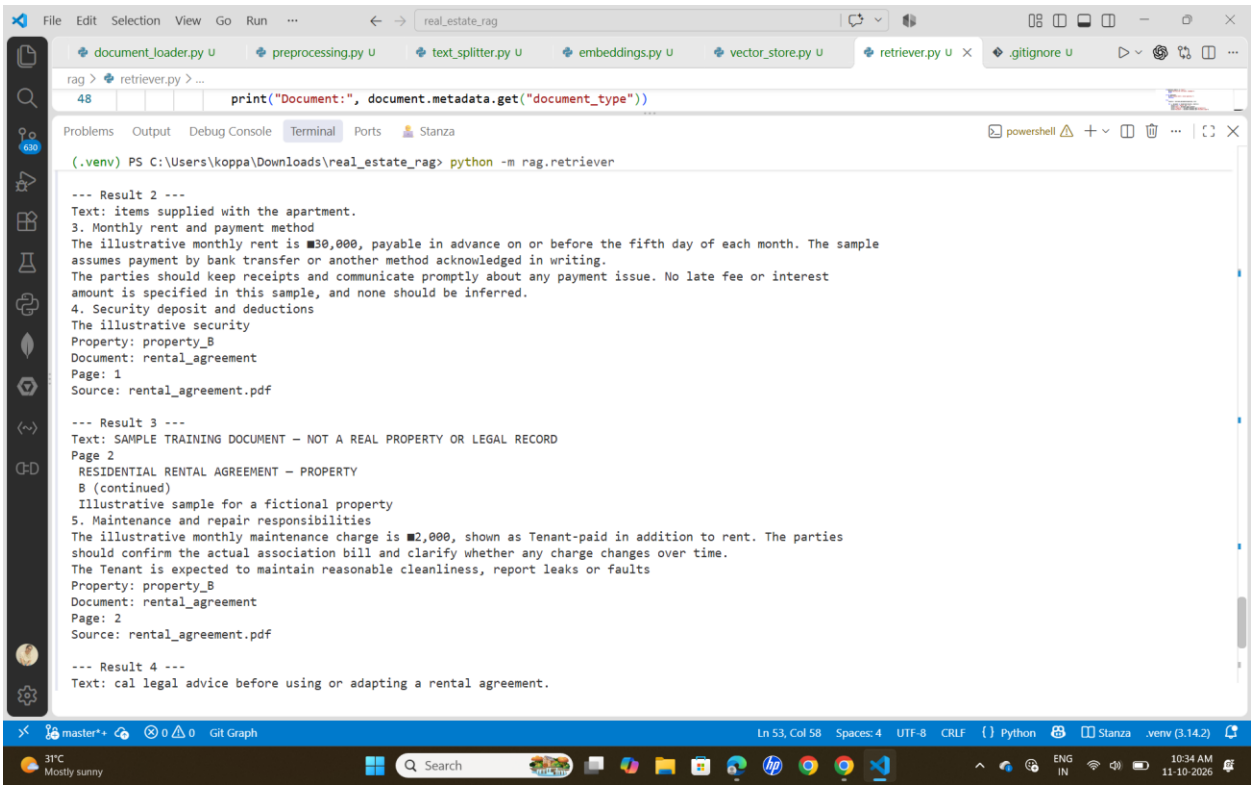
```
Question
   ↓
Embedding
   ↓
FAISS similarity search
   ↓
Top 4 relevant chunks
```

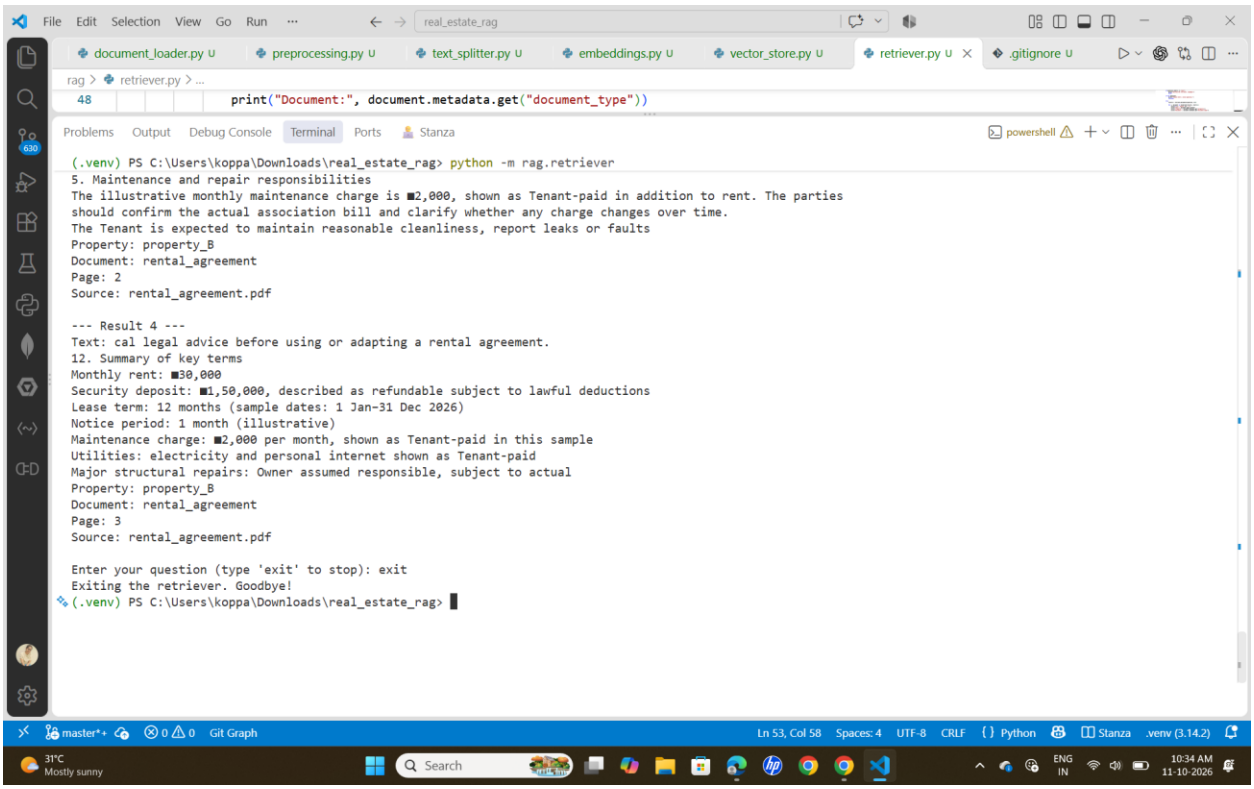
Output:

python -m rag.retriever




File Edit Selection View Go Run
real_estate_rag
document_loader.py U
preprocessing.py U
text_splitter.py U
embeddings.py U
vector_store.py U
retriever.py U
.gitignore U
rag > retriever.py > ...
48 print("Document:", document.metadata.get("document_type"))
Problems Output Debug Console Terminal Ports Stanza
powershell
Page: 3
Source: rental_agreement.pdf
(.venv) PS C:\Users\koppa\Downloads\real_estate_rag> python -m rag.retriever
Enter your question (type 'exit' to stop): What is the monthly rent for Property B?
C:\Users\koppa\Downloads\real_estate_rag\rag\embeddings.py:9: LangChainDeprecationWarning: The class `HuggingFaceEmbeddings` was deprecated in LangChain 0.2.2 and will be removed in 1.0. An updated version of the class exists in the `langchain-huggingface package and should be used instead. To use it run `pip install -U `langchain-huggingface` and import as `from `langchain_huggingface import HuggingFaceEmbeddings``.
embedding_model = HuggingFaceEmbeddings(
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
Loading weights: 100%| 103/103 [00:00<00:00, 2584.32it/s]
FAISS vector database loaded successfully.
Question: What is the monthly rent for Property B?
Relevant chunks retrieved: 4
--- Result 1 ---
Text: vice. This fictional sample does not provide a legal opinion.
12. Summary of key terms
Monthly rent: ■25,000
Security deposit: ■1,00,000, described as refundable subject to lawful deductions
Lease term: 11 months (sample dates: 1 Jan–30 Nov 2026)
Notice period: 2 months (illustrative)
Maintenance charge: ■3,000 per month, shown as Tenant-paid in this sample
Utilities: electricity and personal internet shown as Tenant-paid
Major structural repairs: Owner assumed responsible, subject to actual
Property: property_A
Document: rental_agreement
Page: 3
Source: rental_agreement.pdf
--- Result 2 ---
Text: items supplied with the apartment.
master*+ 0 0 Git Graph
Ln 53, Col 58 Spaces: 4 UTF-8 CRLF Python Stanza .venv (3.14.2)
31°C Mostly sunny
Search
ENG IN
10:33 AM 11-10-2026


File Edit Selection View Go Run
real_estate_rag
document_loader.py U
preprocessing.py U
text_splitter.py U
embeddings.py U
vector_store.py U
retriever.py U
.gitignore U
rag > retriever.py > ...
48 print("Document:", document.metadata.get("document_type"))
Problems Output Debug Console Terminal Ports Stanza
powershell
(.venv) PS C:\Users\koppa\Downloads\real_estate_rag> python -m rag.retriever
--- Result 2 ---
Text: items supplied with the apartment.
3. Monthly rent and payment method
The illustrative monthly rent is ■30,000, payable in advance on or before the fifth day of each month. The sample assumes payment by bank transfer or another method acknowledged in writing.
The parties should keep receipts and communicate promptly about any payment issue. No late fee or interest amount is specified in this sample, and none should be inferred.
4. Security deposit and deductions
The illustrative security
Property: property_B
Document: rental_agreement
Page: 1
Source: rental_agreement.pdf
--- Result 3 ---
Text: SAMPLE TRAINING DOCUMENT — NOT A REAL PROPERTY OR LEGAL RECORD
Page 2
RESIDENTIAL RENTAL AGREEMENT — PROPERTY
B (continued)
Illustrative sample for a fictional property
5. Maintenance and repair responsibilities
The illustrative monthly maintenance charge is ■2,000, shown as Tenant-paid in addition to rent. The parties should confirm the actual association bill and clarify whether any charge changes over time.
The Tenant is expected to maintain reasonable cleanliness, report leaks or faults
Property: property_B
Document: rental_agreement
Page: 2
Source: rental_agreement.pdf
--- Result 4 ---
Text: cal legal advice before using or adapting a rental agreement.
master*+ 0 0 Git Graph
Ln 53, Col 58 Spaces: 4 UTF-8 CRLF Python Stanza .venv (3.14.2)
31°C Mostly sunny
Search
ENG IN
10:34 AM 11-10-2026



12. Step 7 — Gemini 2.5 Flash

Now we use **Gemini 2.5 Flash as the LLM**.

features/qa.py

```
import osimport google.generativeai as genaifrom dotenv import load_dotenvfrom rag.retriever import retrieve_documentsload_dotenv()genai.configure(    api_key=os.getenv("GEMINI_API_KEY"))model = genai.GenerativeModel(    "gemini-2.5-flash")def answer_question(question):    documents = retrieve_documents(        question    )    context = "\n\n".join(        document.page_content        for document in documents    )    prompt = f"""You are a Real Estate Document Intelligence assistant.
```

Output:

Features / qa.py

How your RAG pipeline works now

1. User asks a question

↓

2. Retriever searches FAISS

Finds the four most relevant text chunks

↓

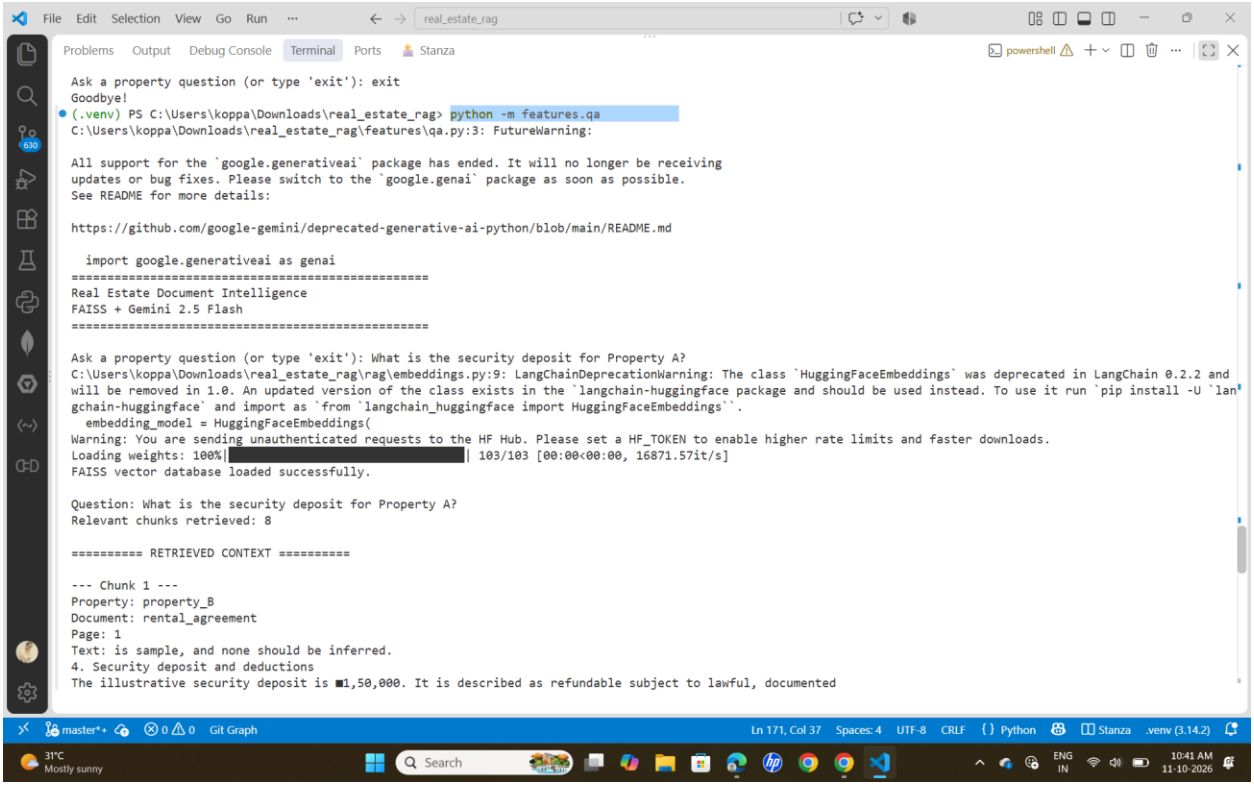
3. Gemini 2.5 Flash

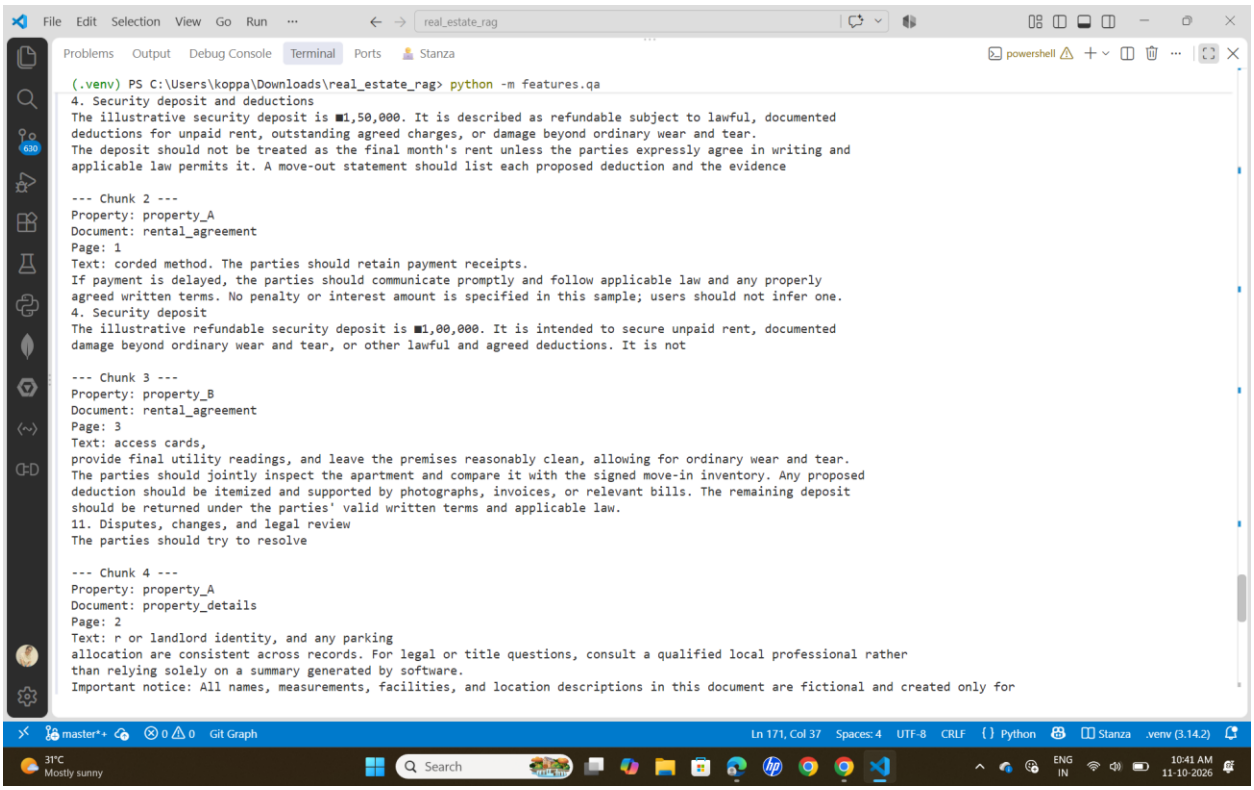
Reads the question and retrieved context

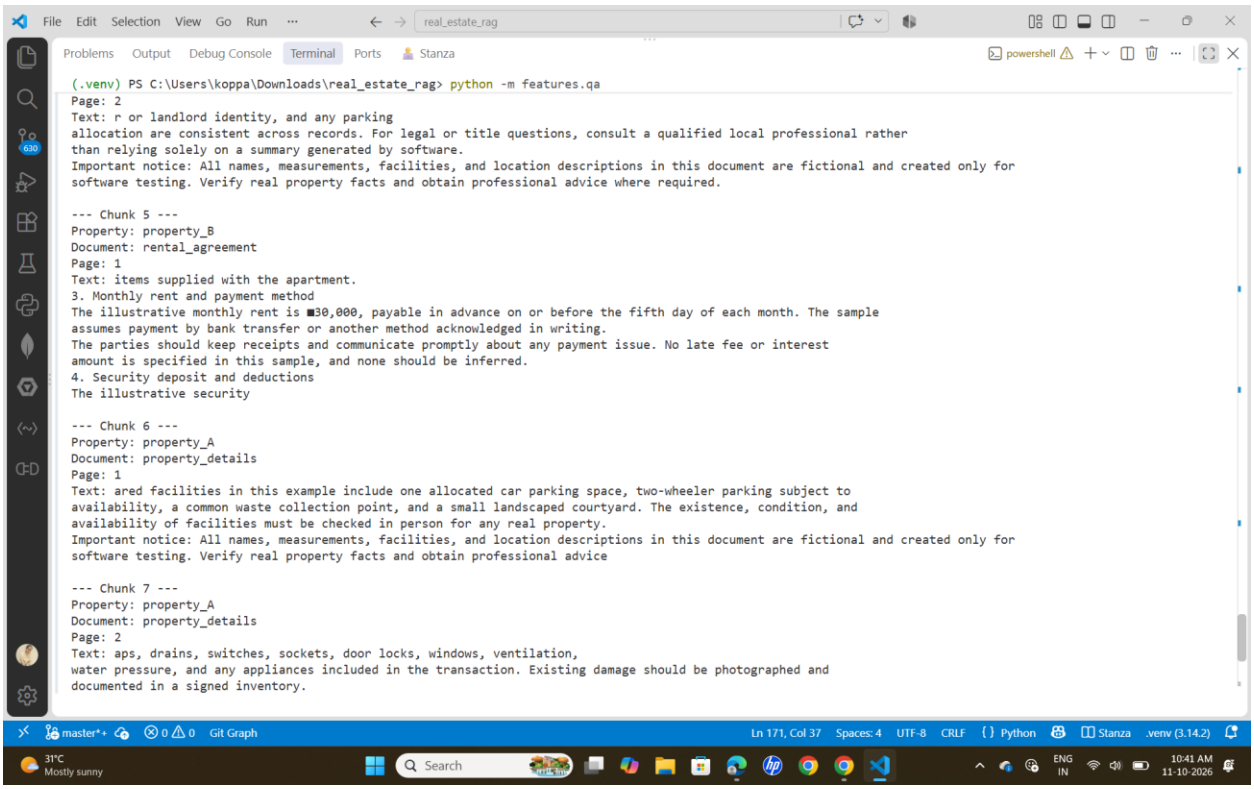
↓

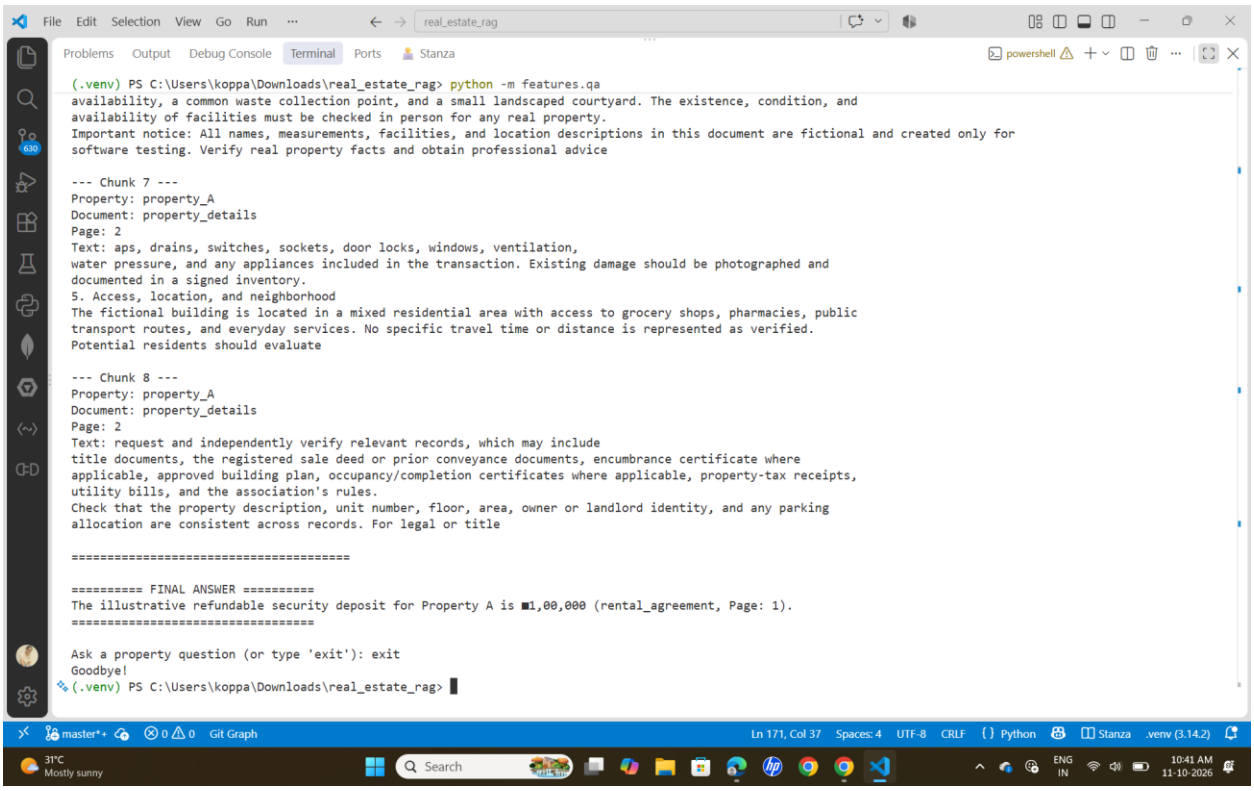
4. Final answer

A natural-language response grounded in your documents


File Edit Selection View Go Run
real_estate_rag
Problems Output Debug Console Terminal Ports Stanza
powershell
Ask a property question (or type 'exit'): exit
Goodbye!
(.venv) PS C:\Users\koppa\Downloads\real_estate_rag> python -m features.qa
C:\Users\koppa\Downloads\real_estate_rag\features\qa.py:3: FutureWarning:
All support for the `google.generativeai` package has ended. It will no longer be receiving
updates or bug fixes. Please switch to the `google.genai` package as soon as possible.
See README for more details:
https://github.com/google-gemini/deprecated-generative-ai-python/blob/main/README.md
import google.generativeai as genai
==
Real Estate Document Intelligence
FAISS + Gemini 2.5 Flash
==
Ask a property question (or type 'exit'): What is the security deposit for Property A?
C:\Users\koppa\Downloads\real_estate_rag\rag\embeddings.py:9: LangChainDeprecationWarning: The class `HuggingFaceEmbeddings` was deprecated in LangChain 0.2.2 and
will be removed in 1.0. An updated version of the class exists in the `langchain-huggingface package and should be used instead. To use it run `pip install -U `lan
gchain-huggingface` and import as `from `langchain_huggingface import HuggingFaceEmbeddings``.
embedding_model = HuggingFaceEmbeddings(
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
Loading weights: 100%| | 103/103 [00:00<00:00, 16871.57it/s]
FAISS vector database loaded successfully.
Question: What is the security deposit for Property A?
Relevant chunks retrieved: 8
========== RETRIEVED CONTEXT ==========
--- Chunk 1 ---
Property: property_B
Document: rental_agreement
Page: 1
Text: is sample, and none should be inferred.
4. Security deposit and deductions
The illustrative security deposit is ■1,50,000. It is described as refundable subject to lawful, documented
master*+ 0 0 Git Graph
Ln 171, Col 37 Spaces: 4 UTF-8 CRLF Python Stanza .venv (3.14.2)
31°C Mostly sunny
Search
ENG IN
10:41 AM 11-10-2026

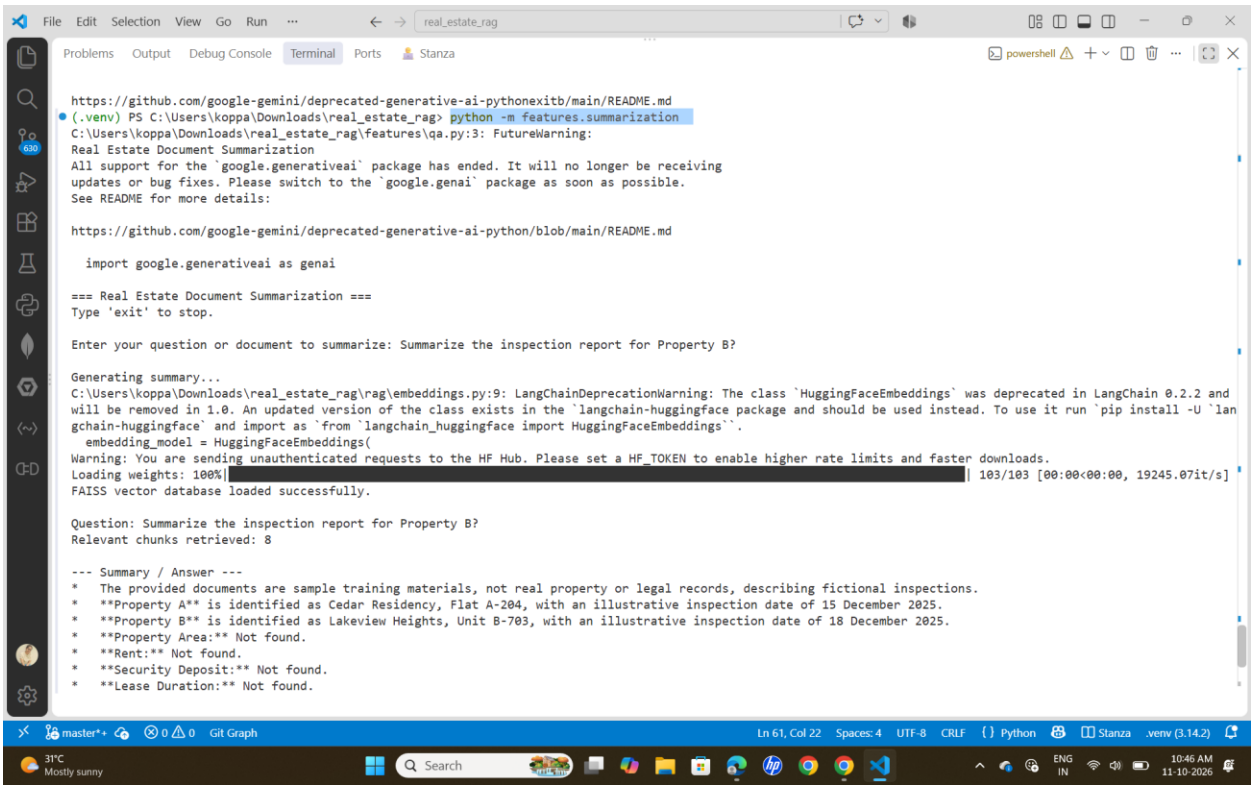

File Edit Selection View Go Run
real_estate_rag
Problems Output Debug Console Terminal Ports Stanza
powershell
(.venv) PS C:\Users\koppa\Downloads\real_estate_rag> python -m features.qa
4. Security deposit and deductions
The illustrative security deposit is ■1,50,000. It is described as refundable subject to lawful, documented
deductions for unpaid rent, outstanding agreed charges, or damage beyond ordinary wear and tear.
The deposit should not be treated as the final month's rent unless the parties expressly agree in writing and
applicable law permits it. A move-out statement should list each proposed deduction and the evidence
--- Chunk 2 ---
Property: property_A
Document: rental_agreement
Page: 1
Text: corded method. The parties should retain payment receipts.
If payment is delayed, the parties should communicate promptly and follow applicable law and any properly
agreed written terms. No penalty or interest amount is specified in this sample; users should not infer one.
4. Security deposit
The illustrative refundable security deposit is ■1,00,000. It is intended to secure unpaid rent, documented
damage beyond ordinary wear and tear, or other lawful and agreed deductions. It is not
--- Chunk 3 ---
Property: property_B
Document: rental_agreement
Page: 3
Text: access cards,
provide final utility readings, and leave the premises reasonably clean, allowing for ordinary wear and tear.
The parties should jointly inspect the apartment and compare it with the signed move-in inventory. Any proposed
deduction should be itemized and supported by photographs, invoices, or relevant bills. The remaining deposit
should be returned under the parties' valid written terms and applicable law.
11. Disputes, changes, and legal review
The parties should try to resolve
--- Chunk 4 ---
Property: property_A
Document: property_details
Page: 2
Text: r or landlord identity, and any parking
allocation are consistent across records. For legal or title questions, consult a qualified local professional rather
than relying solely on a summary generated by software.
Important notice: All names, measurements, facilities, and location descriptions in this document are fictional and created only for
master*+ 0 0 Git Graph
Ln 171, Col 37 Spaces: 4 UTF-8 CRLF Python Stanza .venv (3.14.2)
31°C Mostly sunny
Search
ENG IN
10:41 AM 11-10-2026

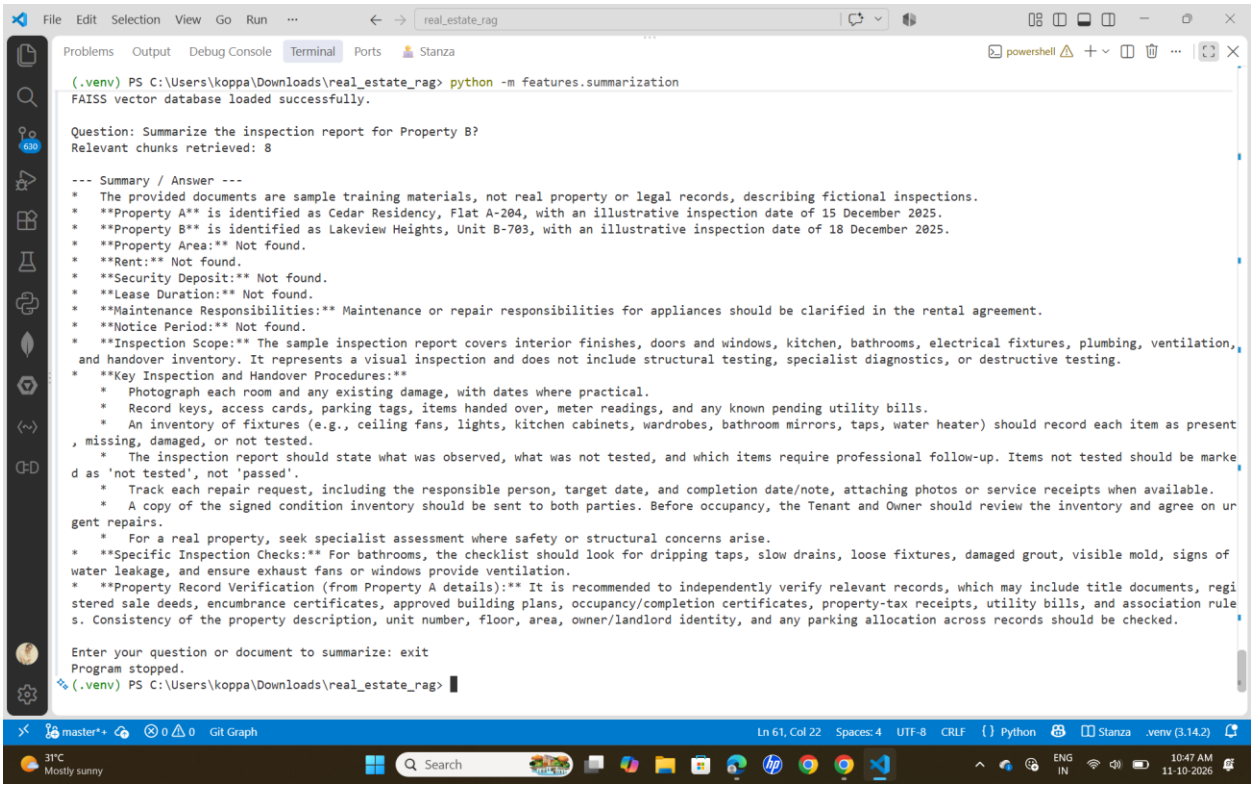




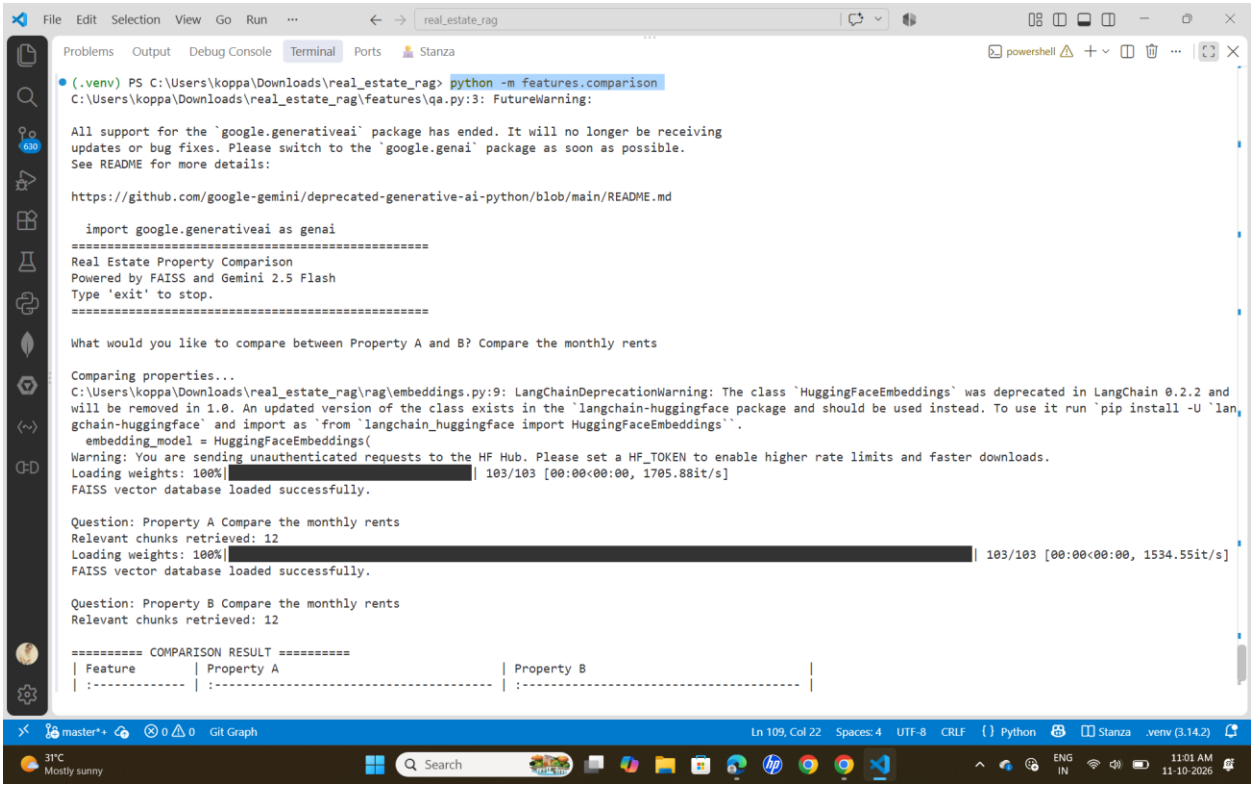
Features / summarazition.py:

python -m features.summarization


File Edit Selection View Go Run
real_estate_rag
Problems Output Debug Console Terminal Ports Stanza
powershell
https://github.com/google-gemini/deprecated-generative-ai-pythonexitb/main/README.md
(.venv) PS C:\Users\koppa\Downloads\real_estate_rag> python -m features.summarization
C:\Users\koppa\Downloads\real_estate_rag\features\qa.py:3: FutureWarning:
Real Estate Document Summarization
All support for the `google.generativeai` package has ended. It will no longer be receiving
updates or bug fixes. Please switch to the `google.genai` package as soon as possible.
See README for more details:
https://github.com/google-gemini/deprecated-generative-ai-python/blob/main/README.md
import google.generativeai as genai
=== Real Estate Document Summarization ===
Type 'exit' to stop.
Enter your question or document to summarize: Summarize the inspection report for Property B?
Generating summary...
C:\Users\koppa\Downloads\real_estate_rag\rag\embeddings.py:9: LangChainDeprecationWarning: The class `HuggingFaceEmbeddings` was deprecated in LangChain 0.2.2 and
will be removed in 1.0. An updated version of the class exists in the `langchain-huggingface package and should be used instead. To use it run `pip install -U `lan
gchain-huggingface` and import as `from `langchain_huggingface import HuggingFaceEmbeddings``.
embedding_model = HuggingFaceEmbeddings(
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
Loading weights: 100%| 103/103 [00:00<00:00, 19245.07it/s]
FAISS vector database loaded successfully.
Question: Summarize the inspection report for Property B?
Relevant chunks retrieved: 8
--- Summary / Answer ---
* The provided documents are sample training materials, not real property or legal records, describing fictional inspections.
* **Property A** is identified as Cedar Residency, Flat A-204, with an illustrative inspection date of 15 December 2025.
* **Property B** is identified as Lakeview Heights, Unit B-703, with an illustrative inspection date of 18 December 2025.
* **Property Area:** Not found.
* **Rent:** Not found.
* **Security Deposit:** Not found.
* **Lease Duration:** Not found.
master*+ 0 0 Git Graph
Ln 61, Col 22 Spaces: 4 UTF-8 CRLF Python Stanza .venv (3.14.2)
31°C Mostly sunny
Search
ENG IN
10:46 AM 11-10-2026


File Edit Selection View Go Run
real_estate_rag
Problems Output Debug Console Terminal Ports Stanza
powershell
(.venv) PS C:\Users\koppa\Downloads\real_estate_rag> python -m features.summarization
FAISS vector database loaded successfully.
Question: Summarize the inspection report for Property B?
Relevant chunks retrieved: 8
--- Summary / Answer ---
* The provided documents are sample training materials, not real property or legal records, describing fictional inspections.
* **Property A** is identified as Cedar Residency, Flat A-204, with an illustrative inspection date of 15 December 2025.
* **Property B** is identified as Lakeview Heights, Unit B-703, with an illustrative inspection date of 18 December 2025.
* **Property Area:** Not found.
* **Rent:** Not found.
* **Security Deposit:** Not found.
* **Lease Duration:** Not found.
* **Maintenance Responsibilities:** Maintenance or repair responsibilities for appliances should be clarified in the rental agreement.
* **Notice Period:** Not found.
* **Inspection Scope:** The sample inspection report covers interior finishes, doors and windows, kitchen, bathrooms, electrical fixtures, plumbing, ventilation,
and handover inventory. It represents a visual inspection and does not include structural testing, specialist diagnostics, or destructive testing.
* **Key Inspection and Handover Procedures:**
* Photograph each room and any existing damage, with dates where practical.
* Record keys, access cards, parking tags, items handed over, meter readings, and any known pending utility bills.
* An inventory of fixtures (e.g., ceiling fans, lights, kitchen cabinets, wardrobes, bathroom mirrors, taps, water heater) should record each item as present
, missing, damaged, or not tested.
* The inspection report should state what was observed, what was not tested, and which items require professional follow-up. Items not tested should be marke
d as 'not tested', not 'passed'.
* Track each repair request, including the responsible person, target date, and completion date/note, attaching photos or service receipts when available.
* A copy of the signed condition inventory should be sent to both parties. Before occupancy, the Tenant and Owner should review the inventory and agree on ur
gent repairs.
* For a real property, seek specialist assessment where safety or structural concerns arise.
* **Specific Inspection Checks:** For bathrooms, the checklist should look for dripping taps, slow drains, loose fixtures, damaged grout, visible mold, signs of
water leakage, and ensure exhaust fans or windows provide ventilation.
* **Property Record Verification (from Property A details):** It is recommended to independently verify relevant records, which may include title documents, regi
stered sale deeds, encumbrance certificates, approved building plans, occupancy/completion certificates, property-tax receipts, utility bills, and association rule
s. Consistency of the property description, unit number, floor, area, owner/landlord identity, and any parking allocation across records should be checked.
Enter your question or document to summarize: exit
Program stopped.
(.venv) PS C:\Users\koppa\Downloads\real_estate_rag>
master*+ 0 0 Git Graph
Ln 61, Col 22 Spaces: 4 UTF-8 CRLF Python Stanza .venv (3.14.2)
31°C Mostly sunny
Search
ENG IN
10:47 AM 11-10-2026

Features/comparasion.py



```
(.venv) PS C:\Users\koppa\Downloads\real_estate_rag> python -m features.comparison
C:\Users\koppa\Downloads\real_estate_rag\features\qa.py:3: FutureWarning:

All support for the `google.generativeai` package has ended. It will no longer be receiving
updates or bug fixes. Please switch to the `google.genai` package as soon as possible.
See README for more details:

https://github.com/google-gemini/deprecated-generative-ai-python/blob/main/README.md

  import google.generativeai as genai
==================================================
Real Estate Property Comparison
Powered by FAISS and Gemini 2.5 Flash
Type 'exit' to stop.
==================================================

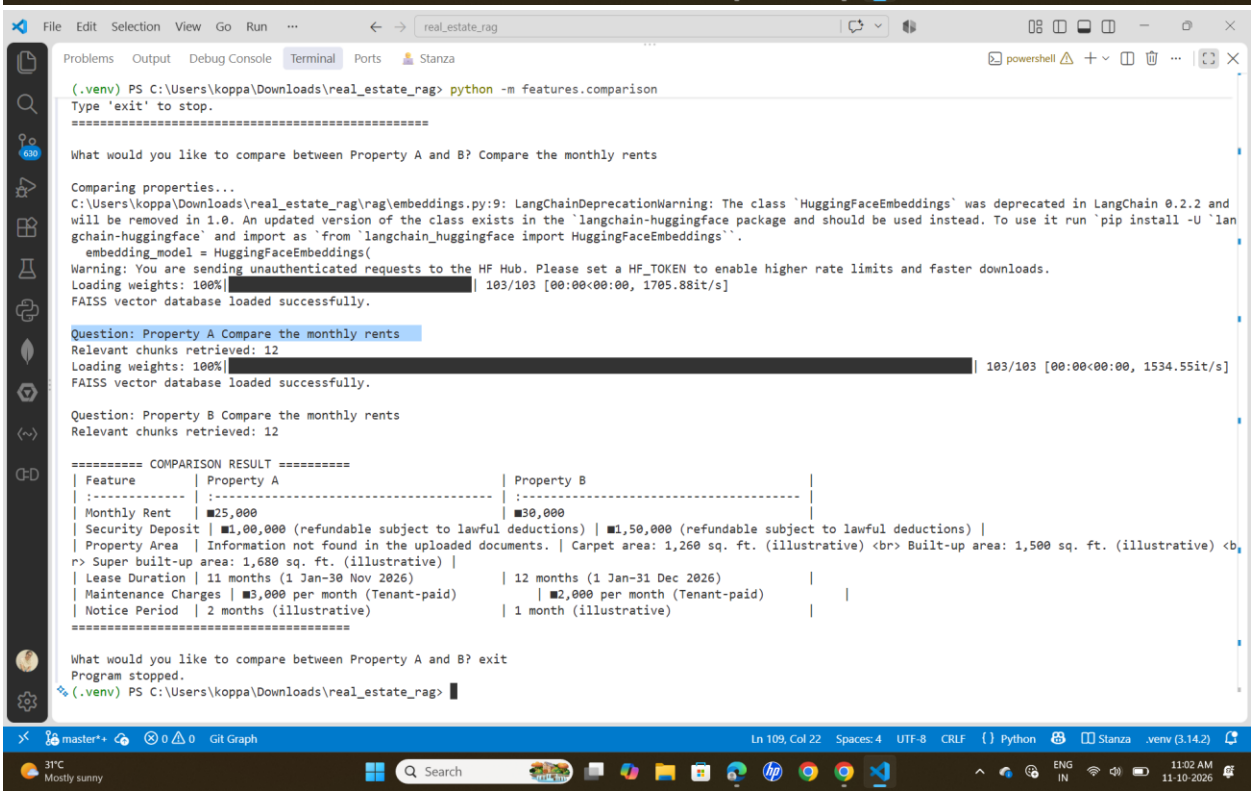
What would you like to compare between Property A and B? Compare the monthly rents

Comparing properties...
C:\Users\koppa\Downloads\real_estate_rag\rag\embeddings.py:9: LangChainDeprecationWarning: The class `HuggingFaceEmbeddings` was deprecated in LangChain 0.2.2 and
will be removed in 1.0. An updated version of the class exists in the `langchain-huggingface package and should be used instead. To use it run `pip install -U `lan
gchain-huggingface` and import as `from `langchain_huggingface import HuggingFaceEmbeddings``.
  embedding_model = HuggingFaceEmbeddings(
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
Loading weights: 100%|| 103/103 [00:00<00:00, 1705.88it/s]
FAISS vector database loaded successfully.

Question: Property A Compare the monthly rents
Relevant chunks retrieved: 12
Loading weights: 100%|| 103/103 [00:00<00:00, 1534.55it/s]
FAISS vector database loaded successfully.

Question: Property B Compare the monthly rents
Relevant chunks retrieved: 12

========== COMPARISON RESULT ==========
| Feature        | Property A                             | Property B                             |
| :------------- | :------------------------------------- | :------------------------------------- |
```



```
(.venv) PS C:\Users\koppa\Downloads\real_estate_rag> python -m features.comparison
Type 'exit' to stop.
==================================================

What would you like to compare between Property A and B? Compare the monthly rents

Comparing properties...
C:\Users\koppa\Downloads\real_estate_rag\rag\embeddings.py:9: LangChainDeprecationWarning: The class `HuggingFaceEmbeddings` was deprecated in LangChain 0.2.2 and
will be removed in 1.0. An updated version of the class exists in the `langchain-huggingface package and should be used instead. To use it run `pip install -U `lan
gchain-huggingface` and import as `from `langchain_huggingface import HuggingFaceEmbeddings``.
  embedding_model = HuggingFaceEmbeddings(
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
Loading weights: 100%|| 103/103 [00:00<00:00, 1705.88it/s]
FAISS vector database loaded successfully.

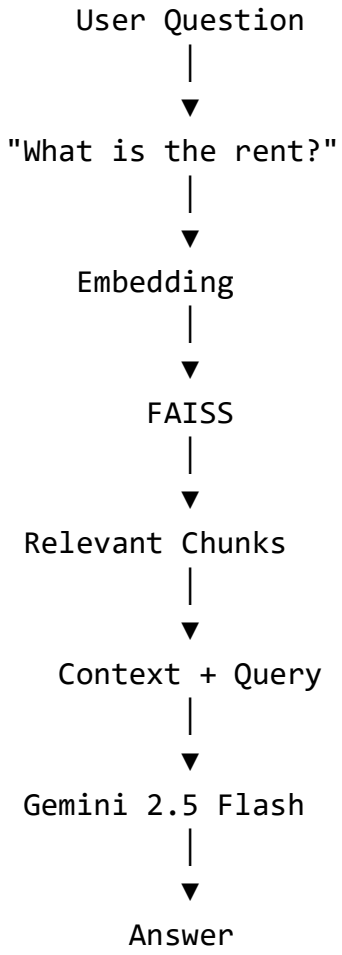
Question: Property A Compare the monthly rents
Relevant chunks retrieved: 12
Loading weights: 100%|| 103/103 [00:00<00:00, 1534.55it/s]
FAISS vector database loaded successfully.

Question: Property B Compare the monthly rents
Relevant chunks retrieved: 12

========== COMPARISON RESULT ==========
| Feature        | Property A                             | Property B                             |
| :------------- | :------------------------------------- | :------------------------------------- |
| Monthly Rent   | ■25,000                                | ■30,000                                |
| Security Deposit | ■1,00,000 (refundable subject to lawful deductions) | ■1,50,000 (refundable subject to lawful deductions) |
| Property Area  | Information not found in the uploaded documents. | Carpet area: 1,260 sq. ft. (illustrative) <br> Built-up area: 1,500 sq. ft. (illustrative) <b
r> Super built-up area: 1,680 sq. ft. (illustrative) |
| Lease Duration | 11 months (1 Jan–30 Nov 2026)          | 12 months (1 Jan–31 Dec 2026)          |
| Maintenance Charges | ■3,000 per month (Tenant-paid)    | ■2,000 per month (Tenant-paid)         |
| Notice Period  | 2 months (illustrative)                | 1 month (illustrative)                 |
==========

What would you like to compare between Property A and B? exit
Program stopped.
(.venv) PS C:\Users\koppa\Downloads\real_estate_rag>
```

13. Your Q&A pipeline is now



This is your actual **RAG pipeline**.

14. Step 8 — Prompt file

`rag/prompts.py`

```
QA_PROMPT = """You are a Real Estate Document Intelligence
assistant.Use ONLY the provided document
context.Context:{context}Question:{question}Rules:1. Do not invent
information.2. Do not use outside knowledge.3. If the answer is
```

```
unavailable, say:   "Information not found in the uploaded
documents."4. Give a concise answer."""
```

Later we will create:

SUMMARY_PROMPT

and:

COMPARISON_PROMPT

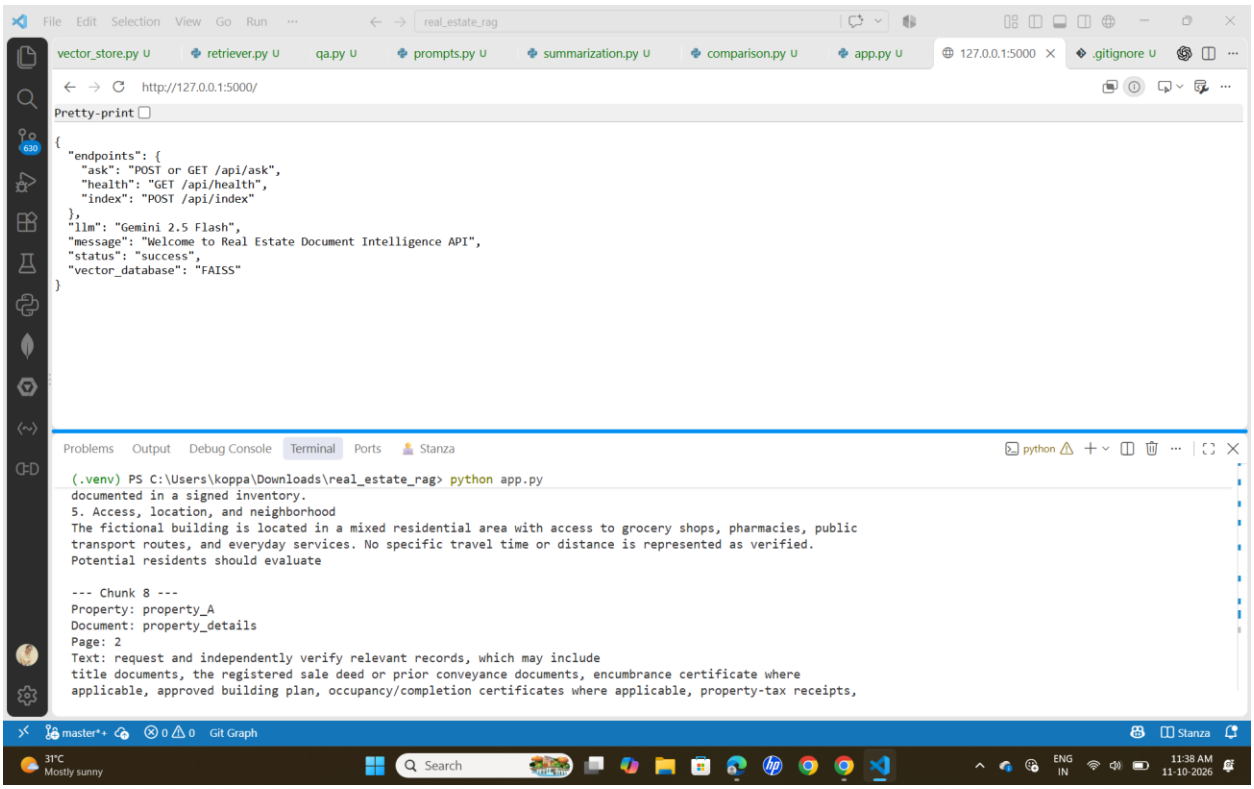
inside the same file.

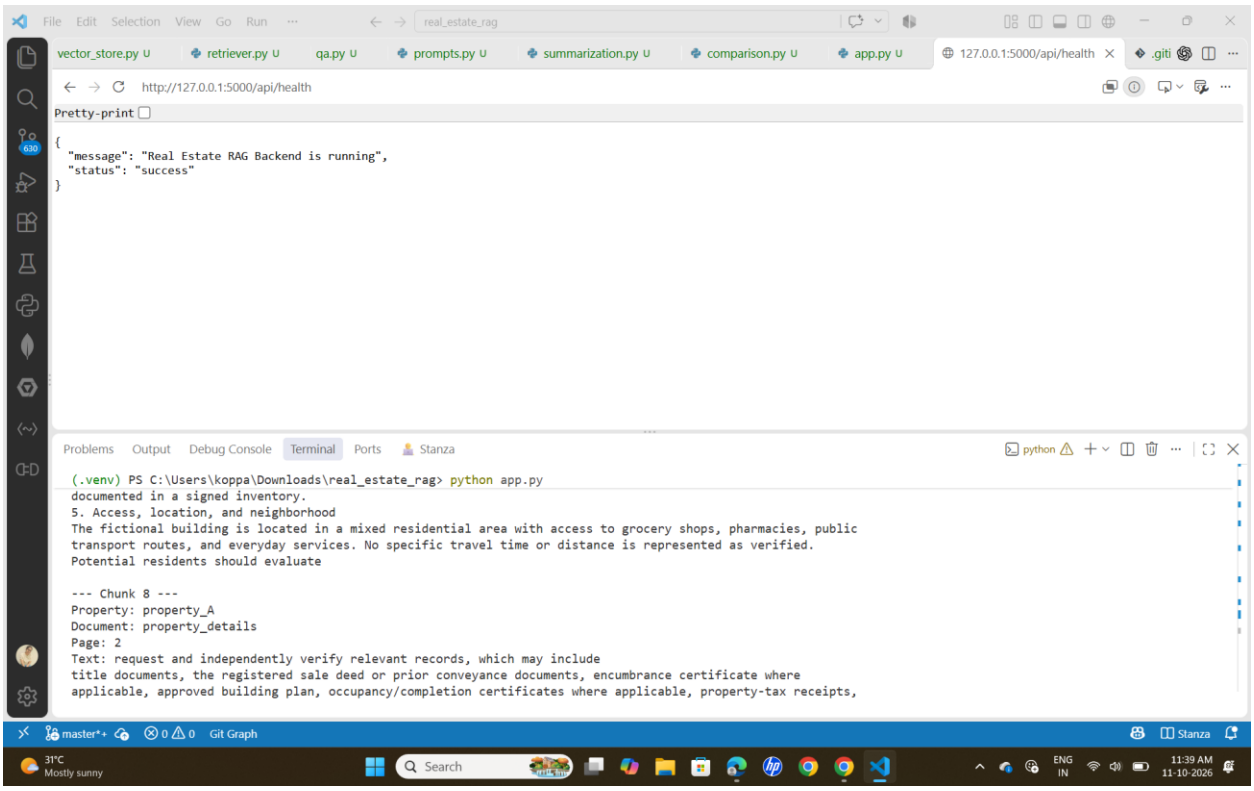
15. Step 9 — Flask backend

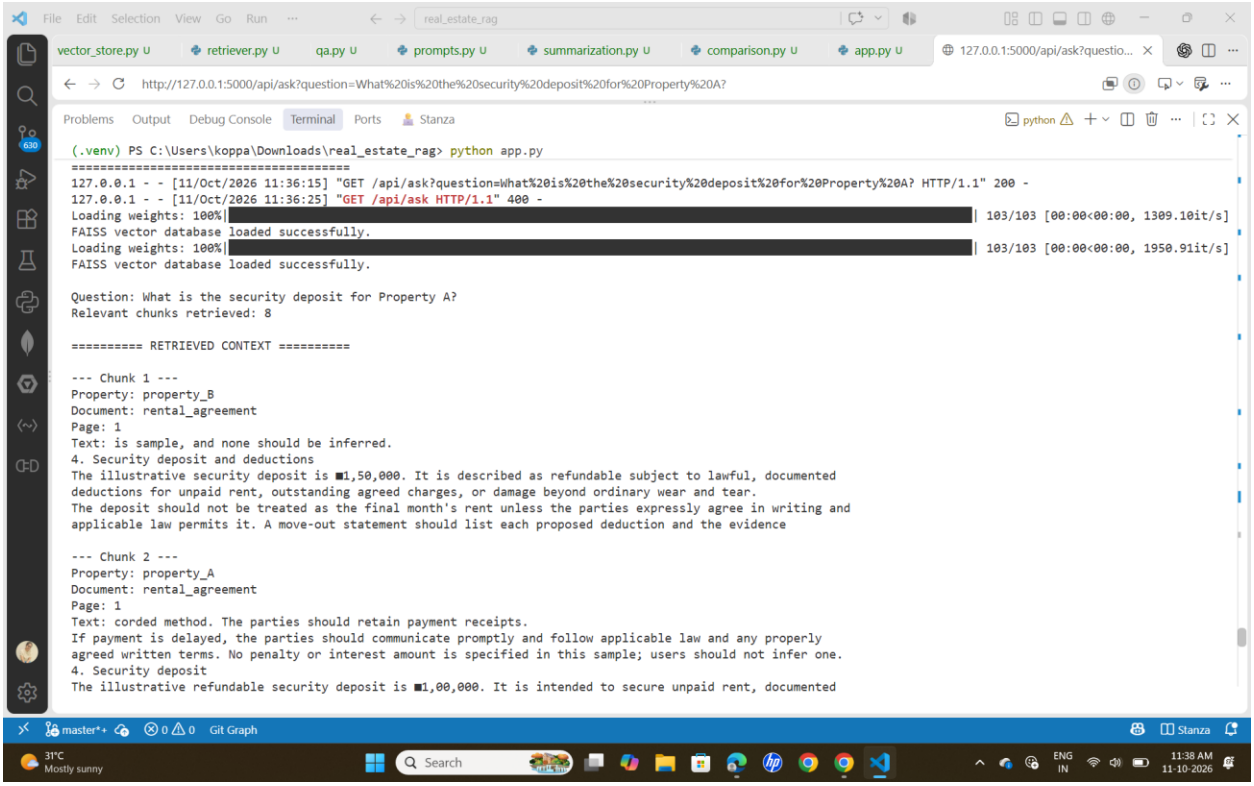
app.py

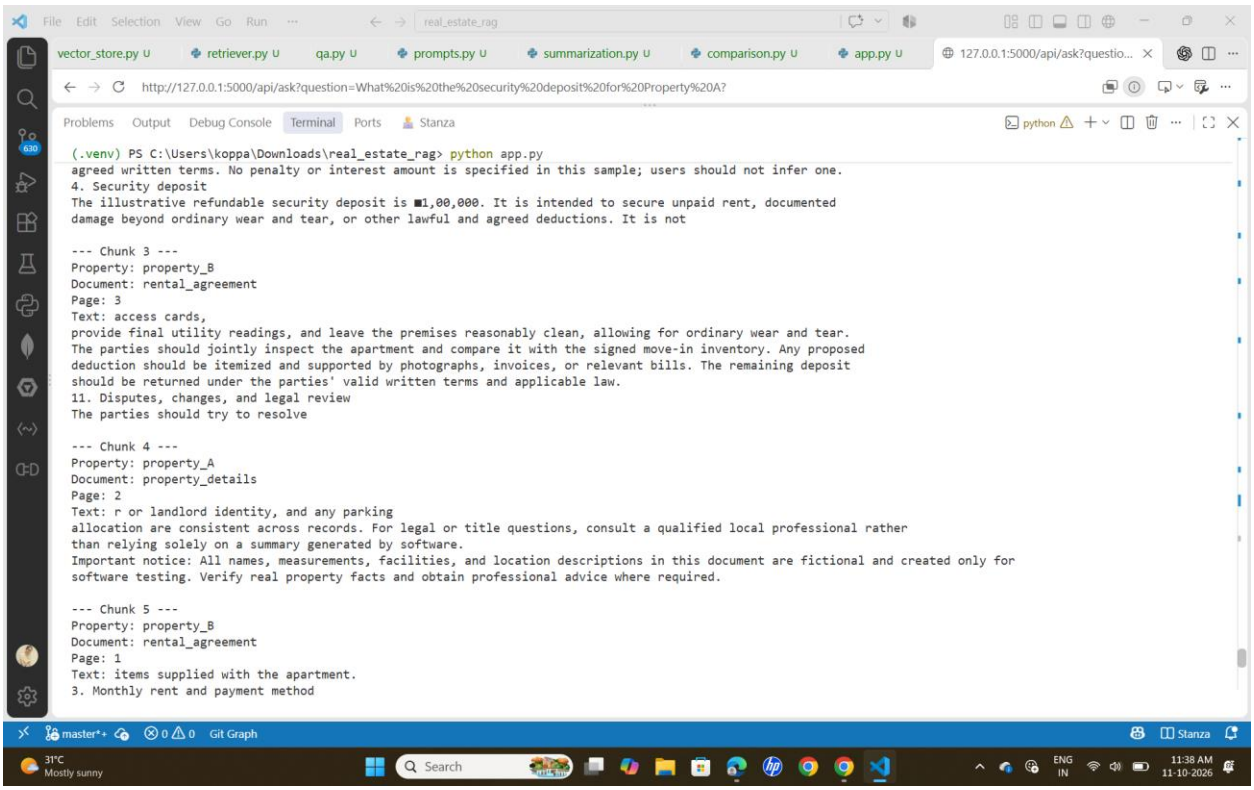
```
from flask import Flask, jsonify, requestfrom flask_cors import
CORSfrom dotenv import load_dotenvfrom rag.document_loader import
load_all_documentsfrom rag.text_splitter import split_documentsfrom
rag.vector_store import create_vector_storefrom features.qa import
answer_questionload_dotenv()app =
Flask(__name__)CORS(app)@app.route(    "/api/health",
methods=["GET"])def health():    return jsonify({        "status":
"success",        "message": "Real Estate RAG Backend is
running"    })@app.route(    "/api/index",    methods=["POST"])def
index_documents():
```

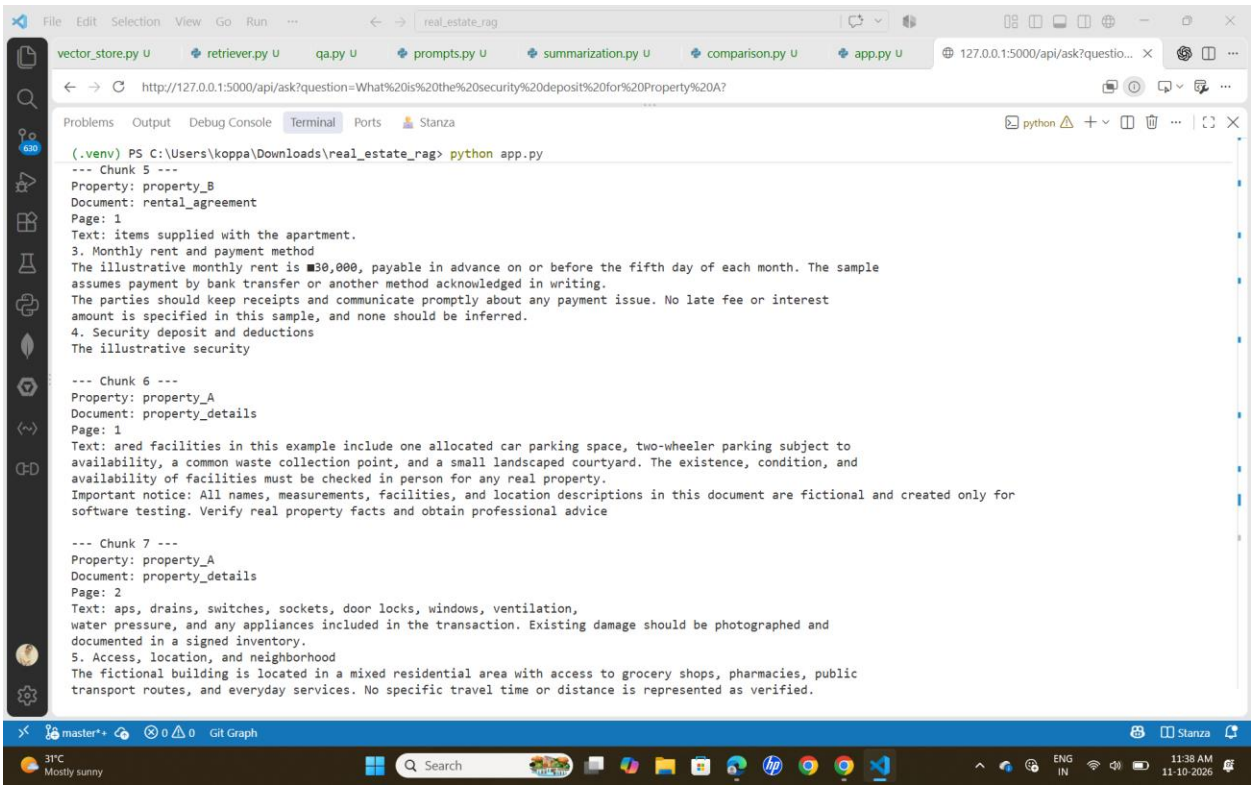
OUTPUT:

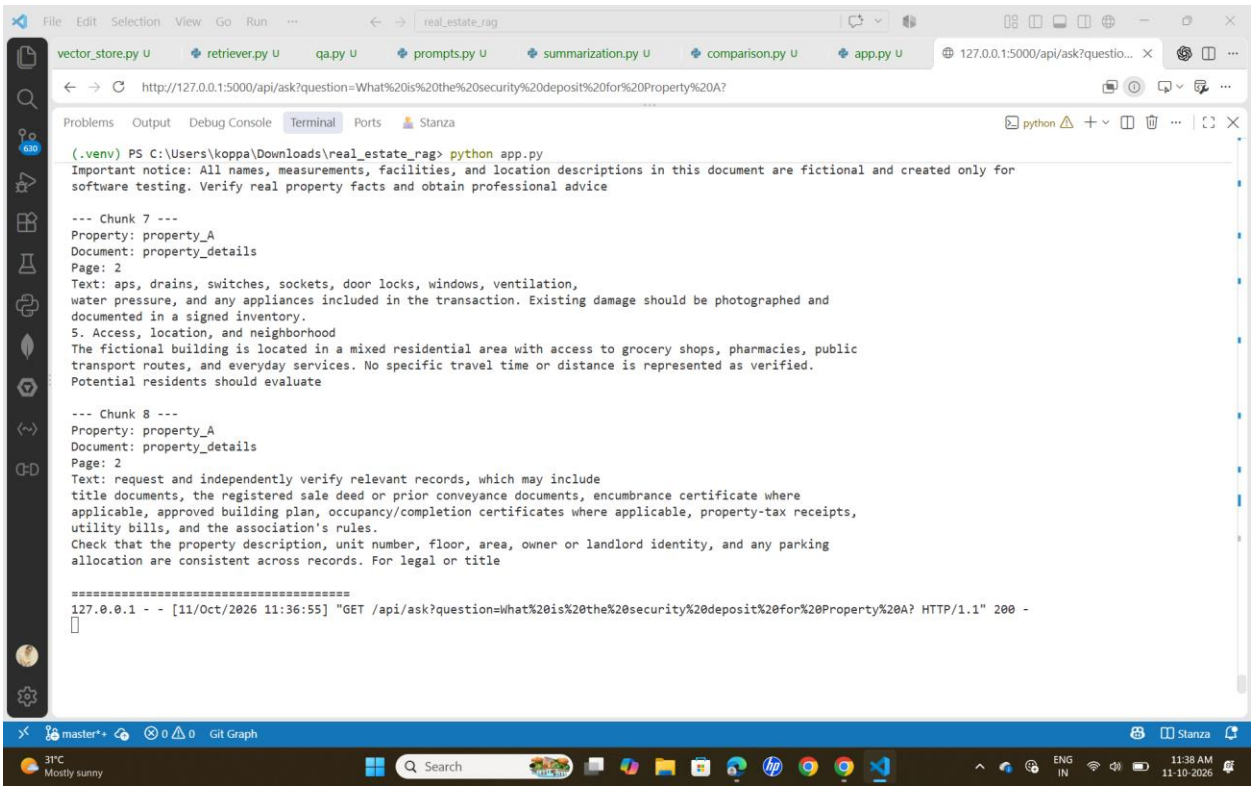

vector_store.py
retriever.py
qa.py
prompts.py
summarization.py
comparison.py
app.py
127.0.0.1:5000
.gitignore
http://127.0.0.1:5000/
Pretty-print
{
"endpoints": {
"ask": "POST or GET /api/ask",
"health": "GET /api/health",
"index": "POST /api/index"
},
"llm": "Gemini 2.5 Flash",
"message": "Welcome to Real Estate Document Intelligence API",
"status": "success",
"vector_database": "FAISS"
}
Problems
Output
Debug Console
Terminal
Ports
Stanza
(.venv) PS C:\Users\koppa\Downloads\real_estate_rag> python app.py
documented in a signed inventory.
5. Access, location, and neighborhood
The fictional building is located in a mixed residential area with access to grocery shops, pharmacies, public
transport routes, and everyday services. No specific travel time or distance is represented as verified.
Potential residents should evaluate
--- Chunk 8 ---
Property: property_A
Document: property_details
Page: 2
Text: request and independently verify relevant records, which may include
title documents, the registered sale deed or prior conveyance documents, encumbrance certificate where
applicable, approved building plan, occupancy/completion certificates where applicable, property-tax receipts,
master*+
Git Graph
31°C
Mostly sunny
Search
ENG
IN
11:38 AM
11-10-2026


vector_store.py
retriever.py
qa.py
prompts.py
summarization.py
comparison.py
app.py
127.0.0.1:5000/api/health
http://127.0.0.1:5000/api/health
Pretty-print
{
"message": "Real Estate RAG Backend is running",
"status": "success"
}
Problems
Output
Debug Console
Terminal
Ports
Stanza
(.venv) PS C:\Users\koppa\Downloads\real_estate_rag> python app.py
documented in a signed inventory.
5. Access, location, and neighborhood
The fictional building is located in a mixed residential area with access to grocery shops, pharmacies, public
transport routes, and everyday services. No specific travel time or distance is represented as verified.
Potential residents should evaluate
--- Chunk 8 ---
Property: property_A
Document: property_details
Page: 2
Text: request and independently verify relevant records, which may include
title documents, the registered sale deed or prior conveyance documents, encumbrance certificate where
applicable, approved building plan, occupancy/completion certificates where applicable, property-tax receipts,
master*+
Git Graph
31°C
Mostly sunny
Search
ENG
IN
11:39 AM
11-10-2026


vector_store.py
retriever.py
qa.py
prompts.py
summarization.py
comparison.py
app.py
http://127.0.0.1:5000/api/ask?question=What%20is%20the%20security%20deposit%20for%20Property%20A?
Problems
Output
Debug Console
Terminal
Ports
Stanza
(.venv) PS C:\Users\koppa\Downloads\real_estate_rag> python app.py
127.0.0.1 - - [11/Oct/2026 11:36:15] "GET /api/ask?question=What%20is%20the%20security%20deposit%20for%20Property%20A? HTTP/1.1" 200 -
127.0.0.1 - - [11/Oct/2026 11:36:25] "GET /api/ask HTTP/1.1" 400 -
Loading weights: 100%| 103/103 [00:00<00:00, 1309.10it/s]
FAISS vector database loaded successfully.
Loading weights: 100%| 103/103 [00:00<00:00, 1950.91it/s]
FAISS vector database loaded successfully.
Question: What is the security deposit for Property A?
Relevant chunks retrieved: 8
========== RETRIEVED CONTEXT ==========
--- Chunk 1 ---
Property: property_B
Document: rental_agreement
Page: 1
Text: is sample, and none should be inferred.
4. Security deposit and deductions
The illustrative security deposit is ■1,50,000. It is described as refundable subject to lawful, documented
deductions for unpaid rent, outstanding agreed charges, or damage beyond ordinary wear and tear.
The deposit should not be treated as the final month's rent unless the parties expressly agree in writing and
applicable law permits it. A move-out statement should list each proposed deduction and the evidence
--- Chunk 2 ---
Property: property_A
Document: rental_agreement
Page: 1
Text: corded method. The parties should retain payment receipts.
If payment is delayed, the parties should communicate promptly and follow applicable law and any properly
agreed written terms. No penalty or interest amount is specified in this sample; users should not infer one.
4. Security deposit
The illustrative refundable security deposit is ■1,00,000. It is intended to secure unpaid rent, documented
master*+
Git Graph
31°C
Mostly sunny
Search
ENG
IN
11:38 AM
11-10-2026


vector_store.py
retriever.py
qa.py
prompts.py
summarization.py
comparison.py
app.py
http://127.0.0.1:5000/api/ask?question=What%20is%20the%20security%20deposit%20for%20Property%20A?
Problems
Output
Debug Console
Terminal
Ports
Stanza
(.venv) PS C:\Users\koppa\Downloads\real_estate_rag> python app.py
agreed written terms. No penalty or interest amount is specified in this sample; users should not infer one.
4. Security deposit
The illustrative refundable security deposit is ■1,00,000. It is intended to secure unpaid rent, documented
damage beyond ordinary wear and tear, or other lawful and agreed deductions. It is not
--- Chunk 3 ---
Property: property_B
Document: rental_agreement
Page: 3
Text: access cards,
provide final utility readings, and leave the premises reasonably clean, allowing for ordinary wear and tear.
The parties should jointly inspect the apartment and compare it with the signed move-in inventory. Any proposed
deduction should be itemized and supported by photographs, invoices, or relevant bills. The remaining deposit
should be returned under the parties' valid written terms and applicable law.
11. Disputes, changes, and legal review
The parties should try to resolve
--- Chunk 4 ---
Property: property_A
Document: property_details
Page: 2
Text: r or landlord identity, and any parking
allocation are consistent across records. For legal or title questions, consult a qualified local professional rather
than relying solely on a summary generated by software.
Important notice: All names, measurements, facilities, and location descriptions in this document are fictional and created only for
software testing. Verify real property facts and obtain professional advice where required.
--- Chunk 5 ---
Property: property_B
Document: rental_agreement
Page: 1
Text: items supplied with the apartment.
3. Monthly rent and payment method
master*+
Git Graph
31°C
Mostly sunny
Search
ENG
IN
11:38 AM
11-10-2026


vector_store.py
retriever.py
qa.py
prompts.py
summarization.py
comparison.py
app.py
http://127.0.0.1:5000/api/ask?question=What%20is%20the%20security%20deposit%20for%20Property%20A?
Problems
Output
Debug Console
Terminal
Ports
Stanza
(.venv) PS C:\Users\koppa\Downloads\real_estate_rag> python app.py
--- Chunk 5 ---
Property: property_B
Document: rental_agreement
Page: 1
Text: items supplied with the apartment.
3. Monthly rent and payment method
The illustrative monthly rent is ■30,000, payable in advance on or before the fifth day of each month. The sample
assumes payment by bank transfer or another method acknowledged in writing.
The parties should keep receipts and communicate promptly about any payment issue. No late fee or interest
amount is specified in this sample, and none should be inferred.
4. Security deposit and deductions
The illustrative security
--- Chunk 6 ---
Property: property_A
Document: property_details
Page: 1
Text: ared facilities in this example include one allocated car parking space, two-wheeler parking subject to
availability, a common waste collection point, and a small landscaped courtyard. The existence, condition, and
availability of facilities must be checked in person for any real property.
Important notice: All names, measurements, facilities, and location descriptions in this document are fictional and created only for
software testing. Verify real property facts and obtain professional advice
--- Chunk 7 ---
Property: property_A
Document: property_details
Page: 2
Text: aps, drains, switches, sockets, door locks, windows, ventilation,
water pressure, and any appliances included in the transaction. Existing damage should be photographed and
documented in a signed inventory.
5. Access, location, and neighborhood
The fictional building is located in a mixed residential area with access to grocery shops, pharmacies, public
transport routes, and everyday services. No specific travel time or distance is represented as verified.
master*+
Git Graph
Search
11:38 AM
11-10-2026


vector_store.py
retriever.py
qa.py
prompts.py
summarization.py
comparison.py
app.py
http://127.0.0.1:5000/api/ask?question=What%20is%20the%20security%20deposit%20for%20Property%20A?
Problems
Output
Debug Console
Terminal
Ports
Stanza
(.venv) PS C:\Users\koppa\Downloads\real_estate_rag> python app.py
Important notice: All names, measurements, facilities, and location descriptions in this document are fictional and created only for
software testing. Verify real property facts and obtain professional advice
--- Chunk 7 ---
Property: property_A
Document: property_details
Page: 2
Text: aps, drains, switches, sockets, door locks, windows, ventilation,
water pressure, and any appliances included in the transaction. Existing damage should be photographed and
documented in a signed inventory.
5. Access, location, and neighborhood
The fictional building is located in a mixed residential area with access to grocery shops, pharmacies, public
transport routes, and everyday services. No specific travel time or distance is represented as verified.
Potential residents should evaluate
--- Chunk 8 ---
Property: property_A
Document: property_details
Page: 2
Text: request and independently verify relevant records, which may include
title documents, the registered sale deed or prior conveyance documents, encumbrance certificate where
applicable, approved building plan, occupancy/completion certificates where applicable, property-tax receipts,
utility bills, and the association's rules.
Check that the property description, unit number, floor, area, owner or landlord identity, and any parking
allocation are consistent across records. For legal or title
==
127.0.0.1 - - [11/Oct/2026 11:36:55] "GET /api/ask?question=What%20is%20the%20security%20deposit%20for%20Property%20A? HTTP/1.1" 200 -
master*+
Git Graph
Search
11:38 AM
11-10-2026

16. Test the backend

Run:

`python app.py`

You should see:

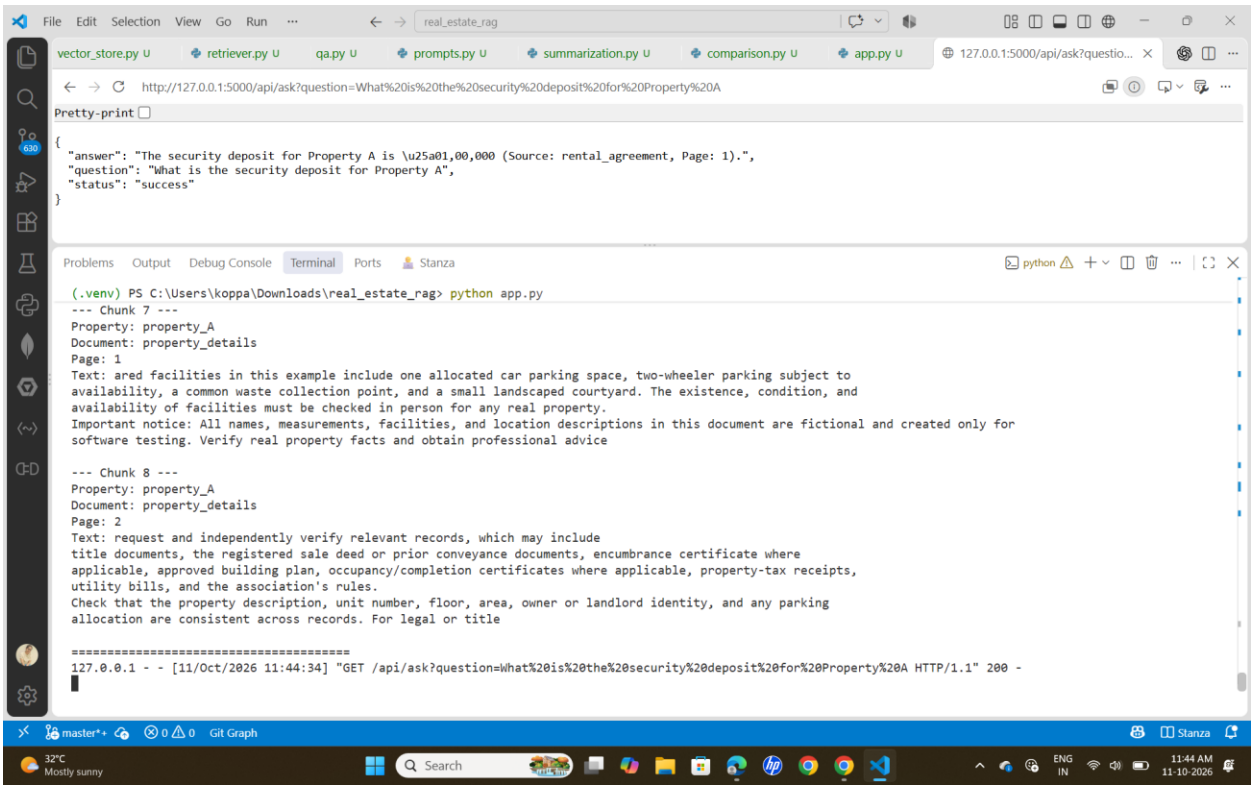
`Running on http://127.0.0.1:5000`

Test 1 — Health

`GET /api/health`

Response:

```
{
    "status": "success",
    "message": "Real Estate RAG Backend is running"
}
```



17. Test 2 — Index documents

Send:

`POST /api/index`

You don't need a body.

Expected:

```
{
    "status": "success",
    "message": "Documents indexed successfully",
    "documents": 1,
    "chunks": 3
}
```

Now your folder should contain:

```
vector_db/
└── faiss_index/
    ├── index.faiss
    └── index.pkl
```

🎯 **That proves your documents have been embedded and stored in FAISS.**

18. Test 3 — Ask a question

Send:

`POST /api/qa`

JSON:

```
{
    "question": "What is the security deposit?"
}
```

Expected:

```
{
    "status": "success",
    "question": "What is the security deposit?",
    "answer": "The security deposit is ₹1,00,000.",
    "sources": [
        {
            "property": "property_A",
            "source": "rental_agreement.pdf",
            "page": 1
        }
    ]
}
```

19. Your technology roles are now very clear

This is important for your project explanation/interview.

PDF

PyPDF

Extracts the document text.

Chunking

`RecursiveCharacterTextSplitter`

Breaks large documents into smaller pieces.

Embeddings

`all-MiniLM-L6-v2`

Converts text into vectors.

Vector database

`FAISS`

Stores and searches those vectors.

Retriever

`FAISS similarity search`

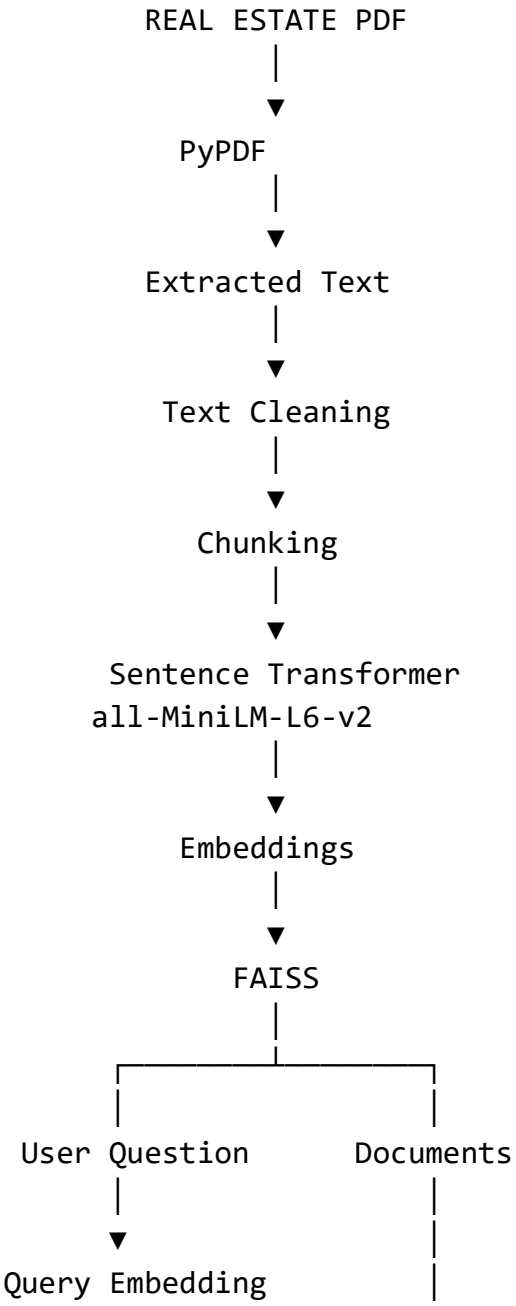
Finds the most relevant chunks.

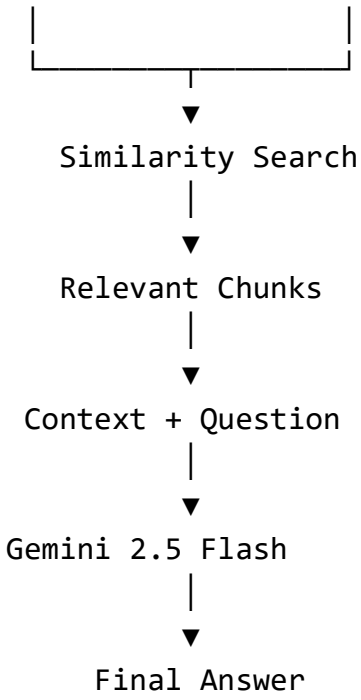
LLM

`Gemini 2.5 Flash`

Uses retrieved context to generate the answer.

20. Complete RAG architecture for YOUR project



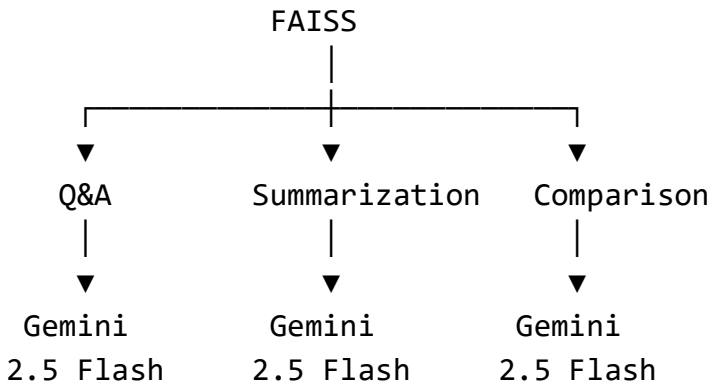


21. And later, the same FAISS database supports all 3 features

You don't create:

```
FAISS_QA
FAISS_SUMMARY
FAISS_COMPARISON
```

Instead:



So **one FAISS knowledge base + Gemini 2.5 Flash** can support the entire application.

For comparison, when we add Property B, the same FAISS index will contain metadata such as:

```
property_A + rental_agreement + page 1
property_A + inspection_report + page 2
property_B + rental_agreement + page 1
property_B + inspection_report + page 2
```

Then we use metadata filtering/retrieval to get Property A and Property B separately before sending both contexts to **Gemini 2.5 Flash**.

One correction from the earlier code

Because you're specifically standardizing on **Gemini 2.5 Flash + FAISS**, use the architecture above consistently and **remove all ChromaDB code/dependencies**. We should build the project incrementally from here rather than mixing the two vector databases.